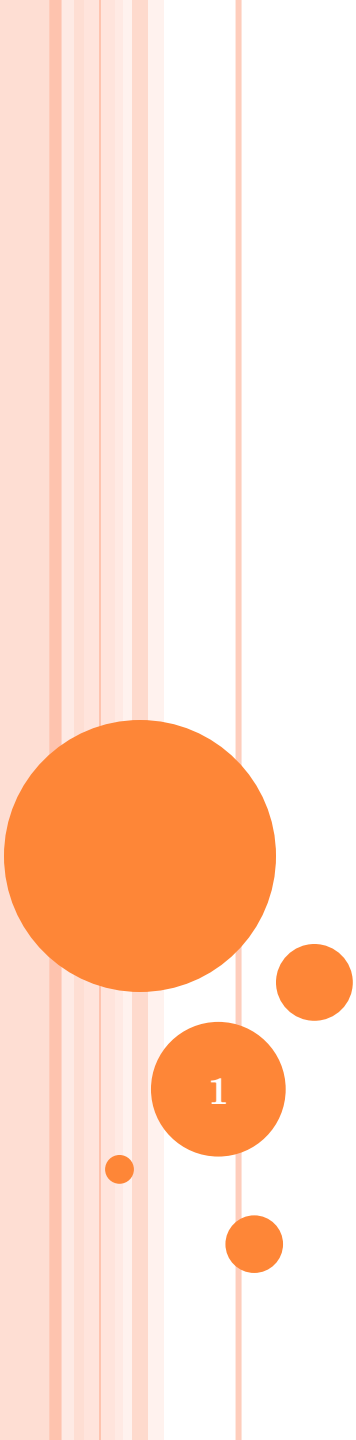




SISTEME DE OPERARE

Curs 12: Gestiunea memoriei (partea 2)

Kinga Márton

Universitatea Tehnică din Cluj-Napoca
Departamentul de Calculatoare

GESTIUNEA MEMORIEI

- În cuprins

- Scurtă **recapitulare** a conceptelor introduse în cursul precedent
- **Concepte noi** legate de **memoria virtuală**
 - Paginare la cerere
 - Copiere la scriere (COW)
 - Algoritmi de înlocuire a paginilor
 - Alocarea cadrelor
 - Thrashing
 - Alte aspecte

MEMORIA – SCURT REZUMAT AL CURSULUI PRECEDENT

- Componentă fundamentală a SO
 - Programul trebuie **încărcat în memorie** pentru a putea fi rulat în cadrul unui proces
- Virtualizarea memoriei
 - Mai multe procese încărcate în memorie
 - Crearea iluziei că fiecare proces are **o memorie dedicată**
 - Spațiu de adrese (cocon în cadrul căruia trăiește procesul)
 - Multiplexare în spațiu
- Gestiunea ierarhiei de sisteme de memorare
 - De la regiștrii până la benzi magnetice
 - **Alocarea, recuperarea, contabilizarea** memoriei
 - Gestiunea **swapping**-ului
- Existența simultană a mai multor procese în memorie
 - Eficiență în utilizarea procesorului
 - Probleme: protecție, relocare, partajare

MEMORIA – SCURT REZUMAT AL CURSULUI PRECEDENT

○ Legarea adreselor

- Adrese fizice (absolute) – adresele locațiilor din memoria fizică
- Adrese logice (virtuale) – adresele generate de procesor
 - Relative la spațiul de adrese al procesului

○ MMU – Memory Management Unit

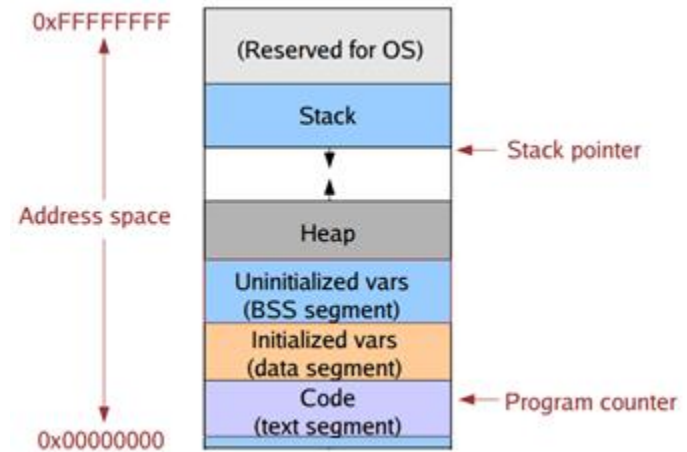
- Componentă a procesorului
- Mecanism HW pentru **maparea adreselor** virtuale la adrese fizice
- Fiecare referință la memorie trece prin MMU

○ Swapping

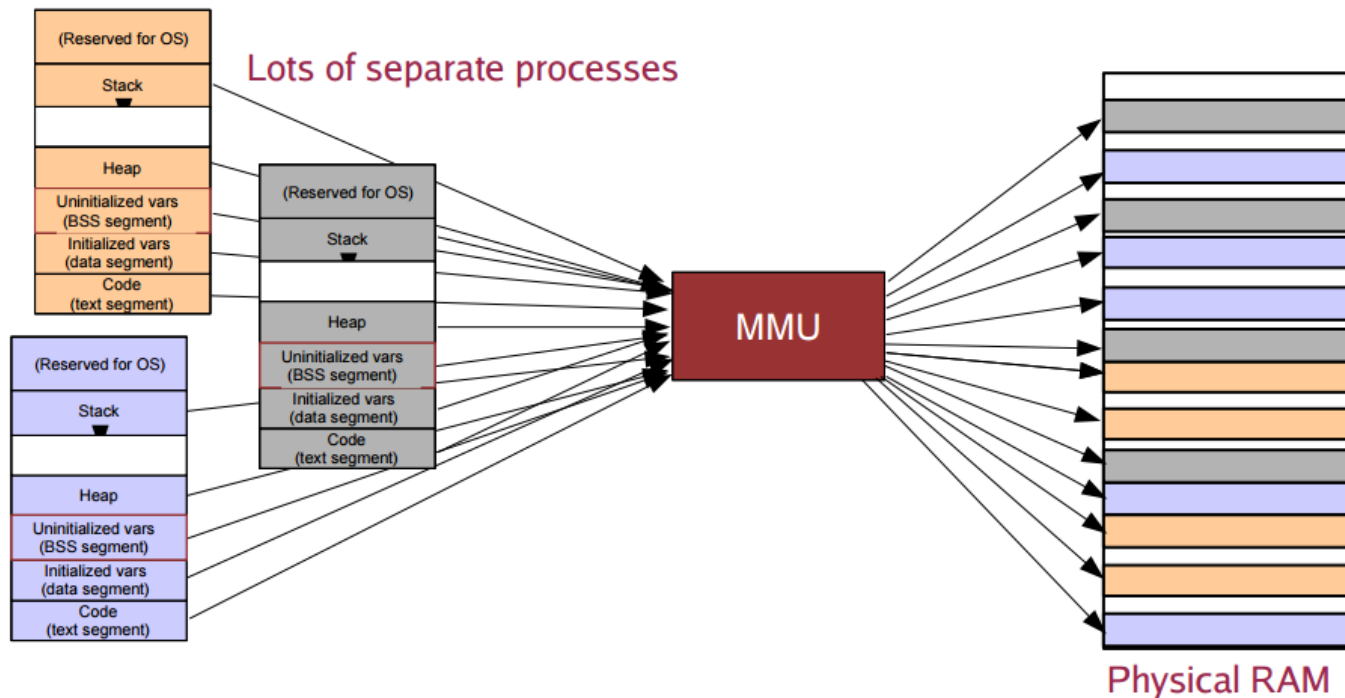
- Memoria internă poate fi insuficientă pentru păstrarea tuturor proceselor în memorie
 - Procesele pot fi **scoase** din memorie
 - **Depozitate** temporar pe disc
 - Ulterior **readuse** în memoria internă pentru continuarea execuției – într-o manieră dinamică

○ Spațiul de adrese al procesului

- Adrese virtuale
- Izolare
- Relocare

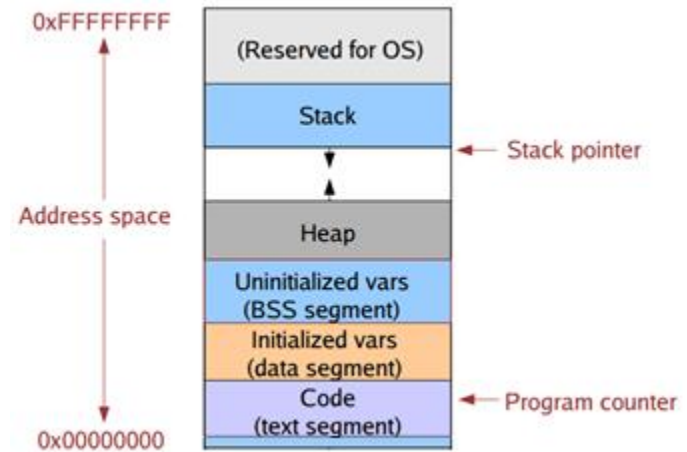


○ Maparea adreselor

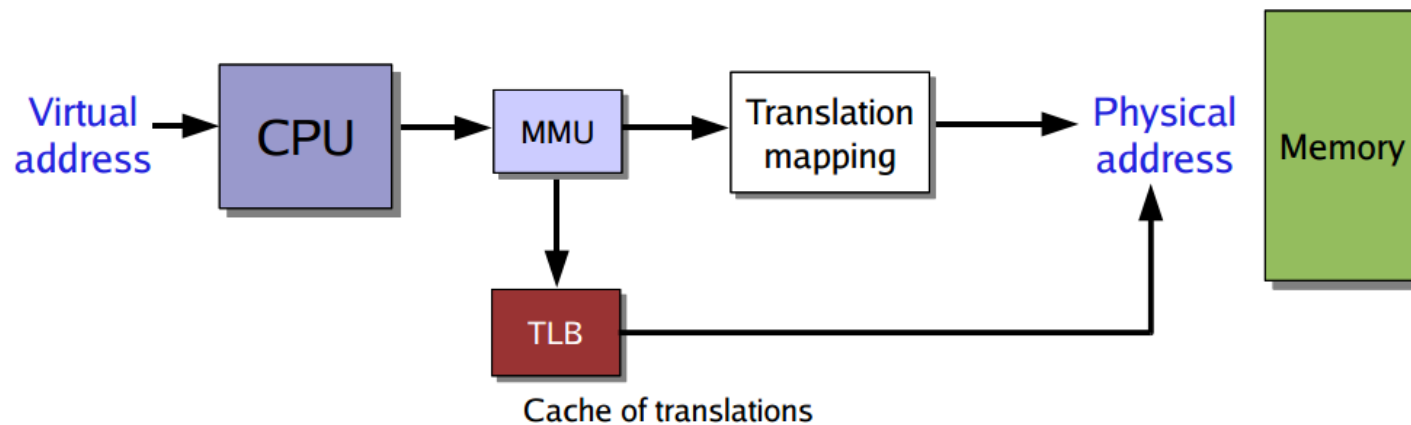


- Spațiul de adrese al procesului

- Adrese virtuale
 - Izolare
 - Relocare



- Maparea adreselor



MEMORIA – SCURT REZUMAT AL CURSULUI PRECEDENT

○ Mecanisme de gestiune a memoriei

○ Alocare **contiguă**

- **Întregul proces** încărcat în memorie într-o **zonă contiguă**

- **Partiționare**

- Partiționare **fixă** – de dimensiuni egale sau diferite
- Partiționare **dinamică** - partiții de dimensiuni variabile

○ Alocare **necontiguă**

- **Întregul proces** încărcat în memorie dar zona **nu trebuie să fie contiguă**

- **Paginare simplă**

- Memoria fizică se împarte în **cadre de pagină** (page frame)
- Memoria logică se împarte în **pagini** - dimensiune fixă
- Tabele de pagini
 - TLB, Paginare ierarhică, Tabele de pagini cu dispersie, Tabele de pagini inversate

- **Segmentare simplă**

- Suport pentru **viziunea utilizatorului** asupra memoriei
- Programul divizat în segmente cu funcționalități specifice și de dimensiuni variate
- Tabela de segmente

○ Fragmentarea

○ Internă

- Se risipesc părți din spațiul alocat

○ Externă

- Se risipesc părți din spațiul nealocat

MEMORIA – SCURT REZUMAT AL CURSULUI PRECEDENT

○ Mecanisme de gestiune a memoriei

○ Alocare **contiguă**

- **Întregul proces** încărcat în memorie într-o **zonă contiguă**

• Partiționare

- Partiționare **fixă** – de dimensiuni egale sau diferite

- Cerințe hardware: **registrul base**

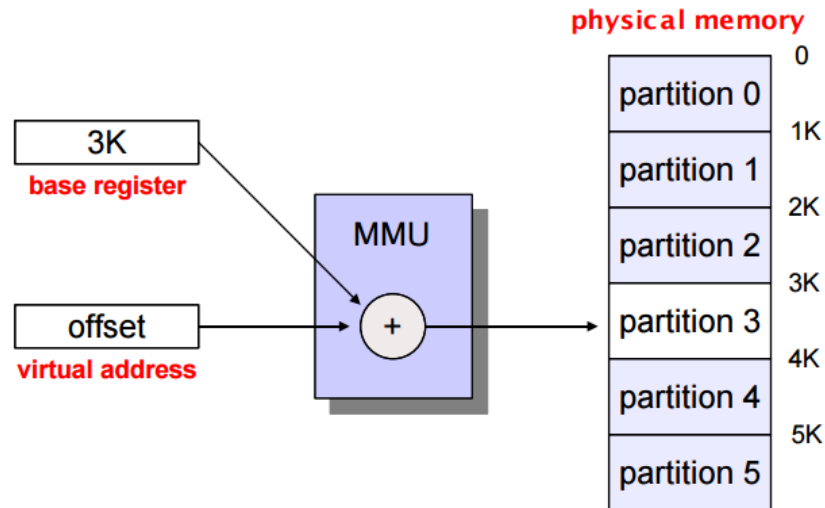
- Translatarea adresei: **adăugarea registrului base la adresa virtuală**

- Avantaje

- Schimbare de context rapidă – doar registrul base trebuie actualizat
- Gestiunea memoriei este simplă: găsirea unei partiții libere

- Dezavantaj

- Fragmentare internă
- Partiții statice – nu e potrivit pentru toate programele



MEMORIA – SCURT REZUMAT AL CURSULUI PRECEDENT

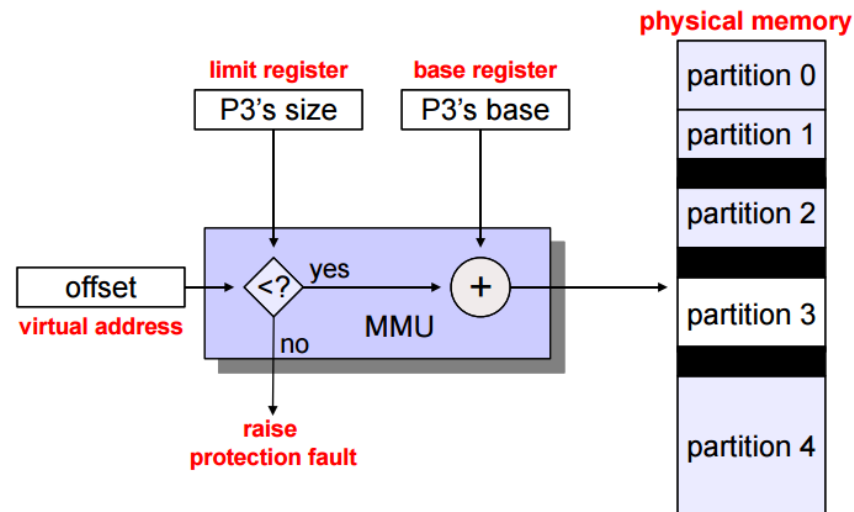
○ Mecanisme de gestiune a memoriei

○ Alocare **contiguă**

- **Întregul proces** încărcat în memorie într-o **zonă contiguă**

• **Partiționare**

- Partiționare **dinamică** - partiții de dimensiuni variabile
 - Cerințe hardware: **registrul base + registrul limit**
 - Rezolvă problema fragmentării interne
 - Apare problema fragmentării externe



MEMORIA – SCURT REZUMAT AL CURSULUI PRECEDENT

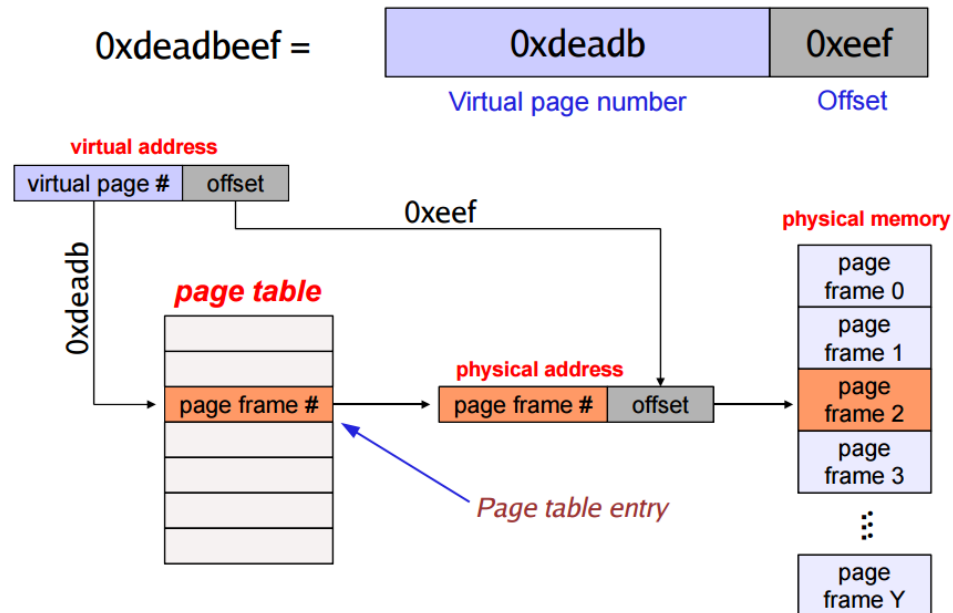
○ Mecanisme de gestiune a memoriei

○ Alocare **necontiguă**

- **Întregul proces** încărcat în memorie dar zona **nu trebuie să fie contiguă**

• **Paginare simplă**

- Memoria fizică se împarte în **cadre de pagină** (page frame)
- Memoria logică se împarte în **pagini** - dimensiune fixă
- Tabele de pagini – stocate în memorie
 - TLB, Paginare ierarhică, Tabele de pagini cu dispersie, Tabele de pagini inversate
- Fragmentare internă - dar redusă



MEMORIA – SCURT REZUMAT AL CURSULUI PRECEDENT

○ Mecanisme de gestiune a memoriei

○ Alocare **necontiguă**

- **Întregul proces** încărcat în memorie dar zona **nu trebuie să fie contiguă**

• **Paginare simplă**

- Memoria fizică se împarte în **cadre de pagină** (page frame)
- Memoria logică se împarte în **pagini** - dimensiune fixă

• **Tabele de pagini**

- TLB, Paginare ierarhică, Tabele de pagini cu dispersie, Tabele de pagini inversate

• **Segmentare simplă**

- Suport pentru **viziunea utilizatorului** asupra memoriei
- Programul divizat în segmente cu funcționalități specifice și de dimensiuni variate

• **Tabela de segmente**

○ Fragmentarea

○ Internă

- Se risipesc părți din spațiul alocat

○ Externă

- Se risipesc părți din spațiul nealocat

MEMORIA VIRTUALĂ

CONTEXT

- Se pot executa doar **instrucțiuni încărcate în memorie**
- Caracteristicile gestiunii memoriei cu paginare sau segmentare simplă
 - Toate adresele generate de procesor sunt **adrese logice** (virtuale)
 - Convertite în mod dinamic la adrese fizice în timpul rulării
 - Procesele pot fi **swapate și readuse în memorie** (posibil la locații diferite) în timpul rulării
 - Procesele pot fi **divizate în elemente** (pagini sau segmente) care nu trebuie neaparat alocate în zone contigue în memorie
 - Translatarea dinamică a adreselor și tabelele de pagini sau segmente permit acesta
- Idee inovatoare – extrasă din caracteristicile de mai sus
 - **NU este nevoie de întregul cod și toate datele pe toată durata execuției**
 - Cod pentru tratarea erorilor, structuri de date mari, etc.
 - Nu este necesar ca toate paginile sau toate segmentele procesului să fie încărcate
 - Posibilitatea de a executa **programe încărcate doar parțial** în memoria internă
 - Dacă următoarea instrucțiune și următoarele date folosite sunt încărcate, execuția poate continua (pentru moment)

EXECUȚIA UNUI PROCES

- SO **încarcă** în memorie câteva elemente (pagini sau segmente) ale programului
 - Cele care conțin **primele instrucțiuni** și primele **date** referite de instrucțiuni
- **Setul rezident**
 - Partea din proces care este efectiv în memoria fizică
- Execuția decurge lin cât timp toate instrucțiunile și datele necesare sunt regăsite în setul rezident
 - Cu ajutorul tabelii de pagini sau a tabelii de segmente se poate determina dacă sunt în setul rezident
- **Întrerupere a execuției**
 - Dacă procesorul ajunge la o **adresă** logică, care **nu este în memoria fizică**
 - Procesul este pus în starea de așteptare (Blocked)
 - SO **aduce în memorie** partea din proces care conține adresa logică referită
 - Operație de I/E – citire de pe disc
 - Pe perioada aceasta planificatorul alege alt proces spre execuție
 - La finalizare procesul este pus înapoi în starea Ready
- Cum? – **Memorie virtuală**
- Eficiența ??? -este posibilă. Detaliere mai târziu

MEMORIA VIRTUALĂ

○ Memorie pe disc

○ Avantaj

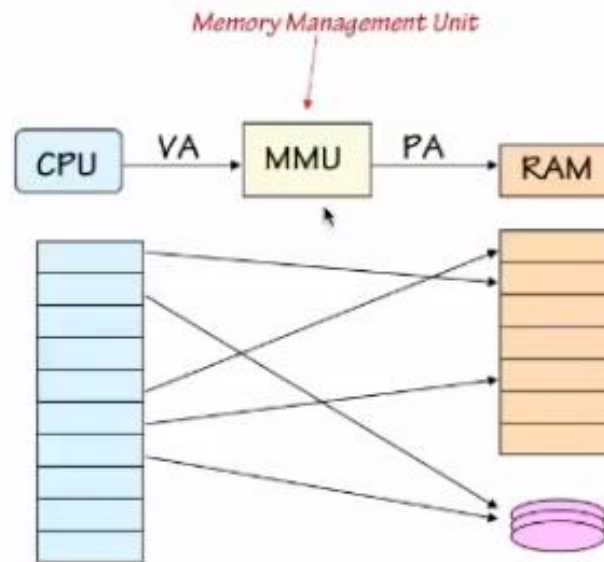
- Programul **nu mai este constrâns de limite**le memoriei fizice
 - Se pot rula programe de dimensiuni mai mari decât memoria fizică
- Doar o **parte din program** trebuie să fie **în memorie** pentru execuție
 - Permite rularea **mai multor procese** în mod concurent
 - Efortul necesar încărcării și depozitării temporare pe disc (swap) este mult diminuat

○ Implică separarea memoriei logice de memoria fizică

- Permite gestiunea spațiului de adrese logic în mod **independent** de locația în memoria fizică
- Permite **crearea** mai ușoară a proceselor noi
 - Copy-on-write
- Permite **partajarea** spațiilor de adrese și a fișierelor de către mai multe procese
 - Partajarea fișierelor – fișiere mapate în memorie

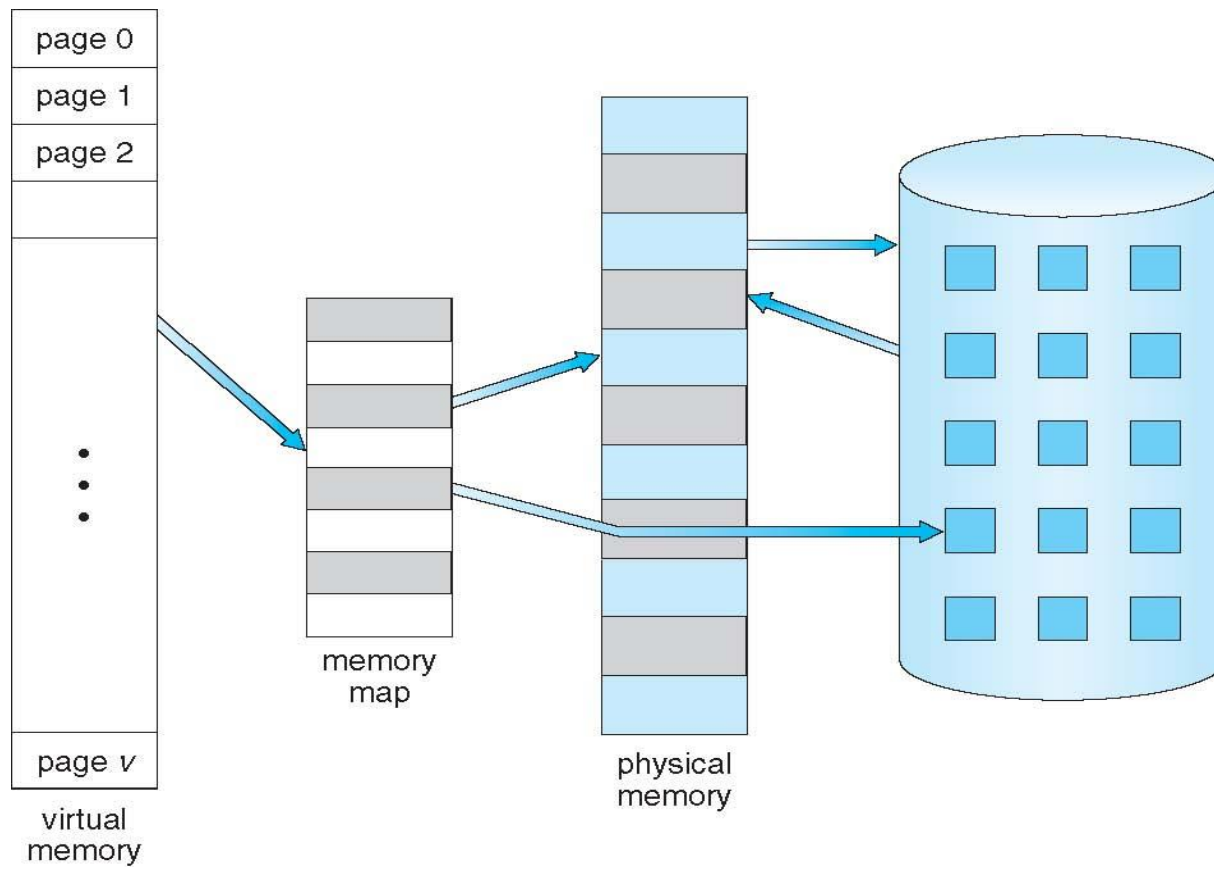
MEMORIA VIRTUALĂ

- Traducerea adreselor



MEMORIA VIRTUALĂ

- Memorie virtuală mai mare decât memoria fizică



PAGINARE

FOLOSIND MEMORIE VIRTUALĂ

PAGINARE

- Termenul de memorie virtuală este asociat de regulă sistemelor care folosesc paginarea
 - Tabele de pagini
 - Fiecare proces are propria tabelă

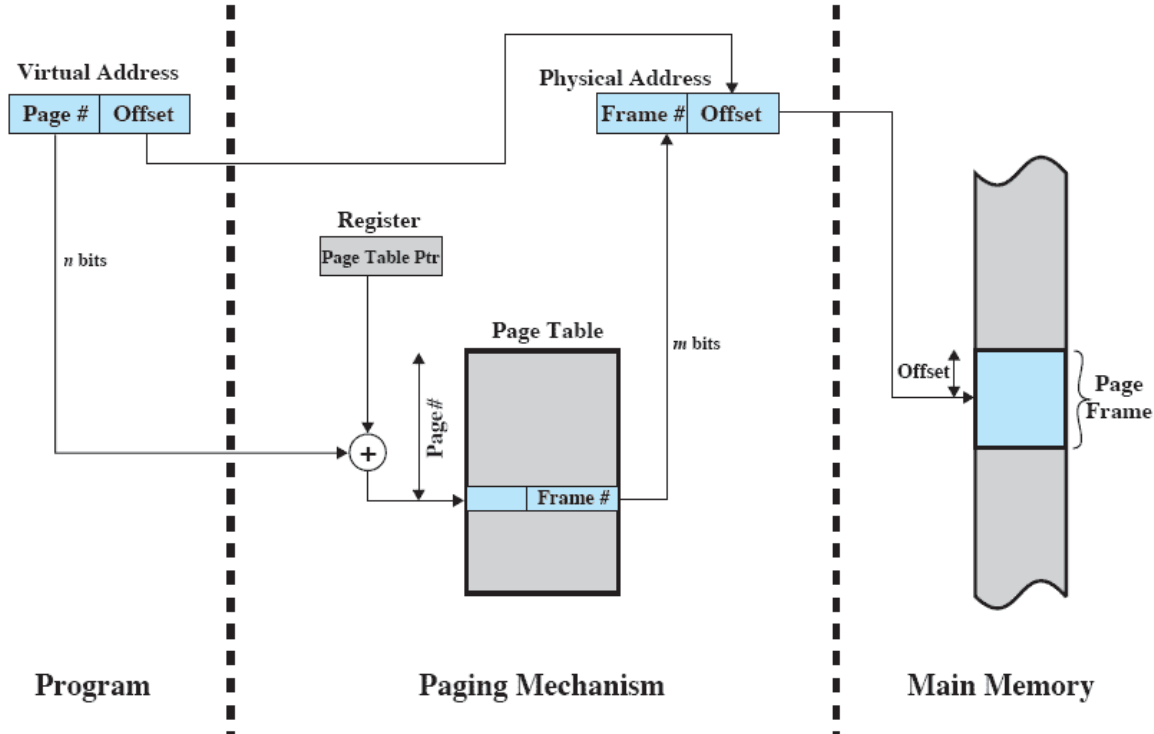


Figure 8.3 Address Translation in a Paging System

PAGINARE

○ Intrare în tabela de pagini

- Este posibil ca **doar unele pagini** să fie încărcate în memorie
- **P** – bit care **indică prezența** acelei pagini în memoria internă
 - Dacă bitul este setat atunci intrarea include și adresa cadrului de pagină din memoria fizică
- **M** – bit care indică dacă s-au efectuat **modificări** asupra conținutului paginii de la ultima încărcare în memoria internă
 - Dacă nu au fost produse modificări pagina nu trebuie scrisă înapoi în memoria virtuală în momentul scoaterii paginii din memoria internă
- Alți biți de control
 - Biți care indică dacă pagina a fost referită recent (util pt mecanismul de înlocuire)
 - Dacă se furnizează partajare și protecție la nivel de pagini, atunci trebuie incluse biți de control și pentru gestiunea acestor aspecte

Virtual Address



Page Table Entry



(a) Paging only

PAGINARE CU MEMORIE VIRTUALĂ ȘI TLB

○ TLB – Translation Lookaside Buffer

- Reducerea accesului dublu la memorie la fiecare conversie de adresă logică
 - Conține asocierile: pagină – cadru, pentru paginile recent folosite
- Memorie asociativă de mare viteză – implementată în hw

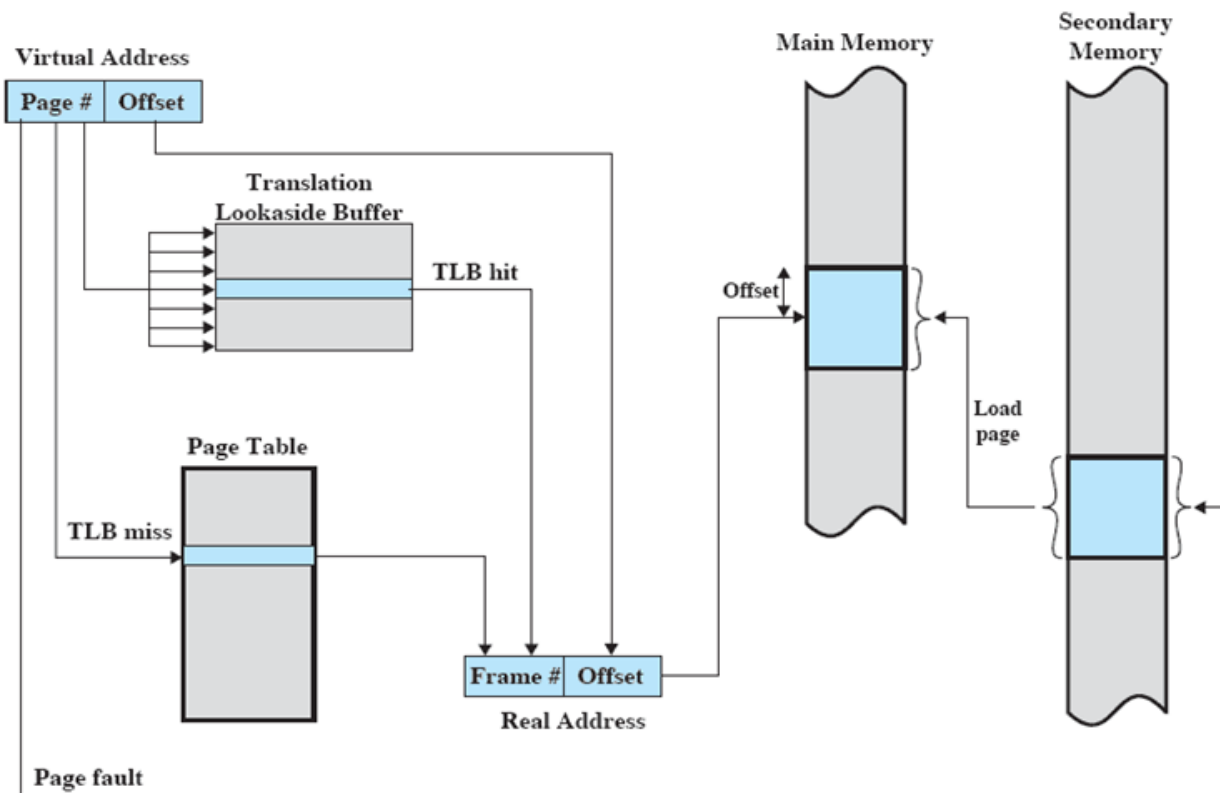
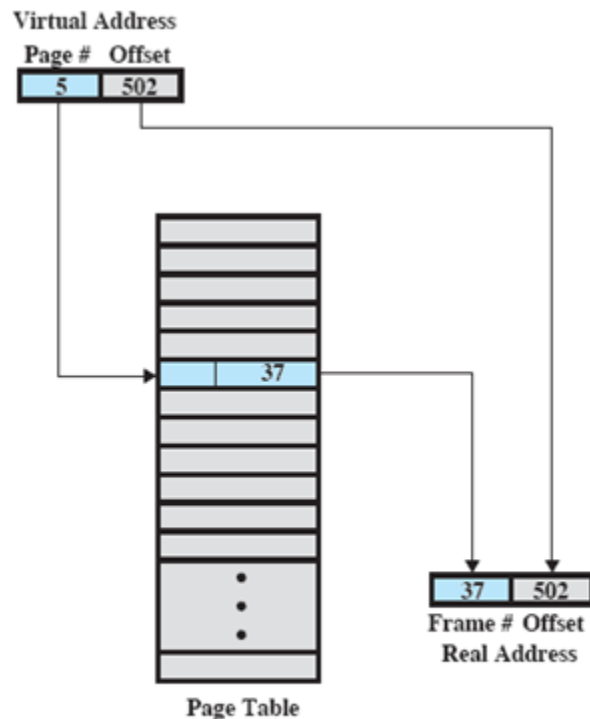


Figure 8.7 Use of a Translation Lookaside Buffer

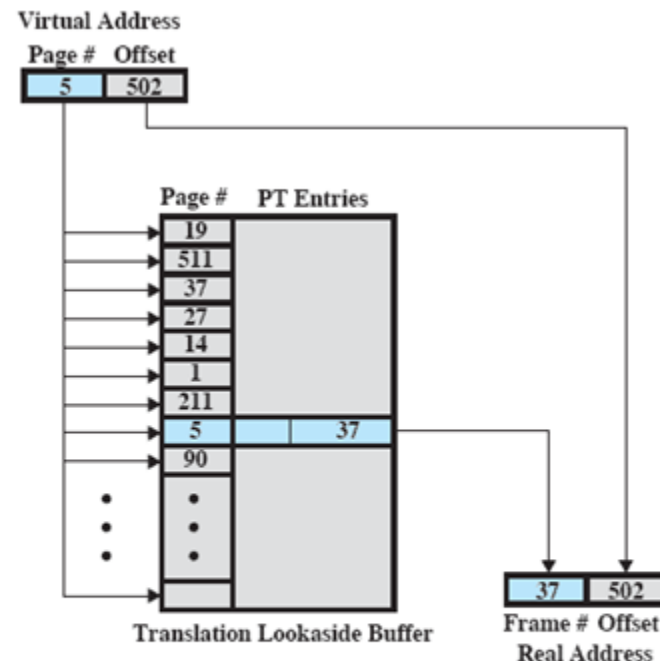
PAGINARE CU MEMORIE VIRTUALĂ ȘI TLB

- TLB – Translation Lookaside Buffer

- o Reducerea accesului dublu la memorie la fiecare conversie de adresă logică
 - Conține asocierile: pagină – cadru, pentru paginile recent folosite
- o Memorie asociativă de mare viteză – implementată în hw



(a) Direct mapping



(b) Associative mapping

PAGING - TLB

Stallings, pg. 352

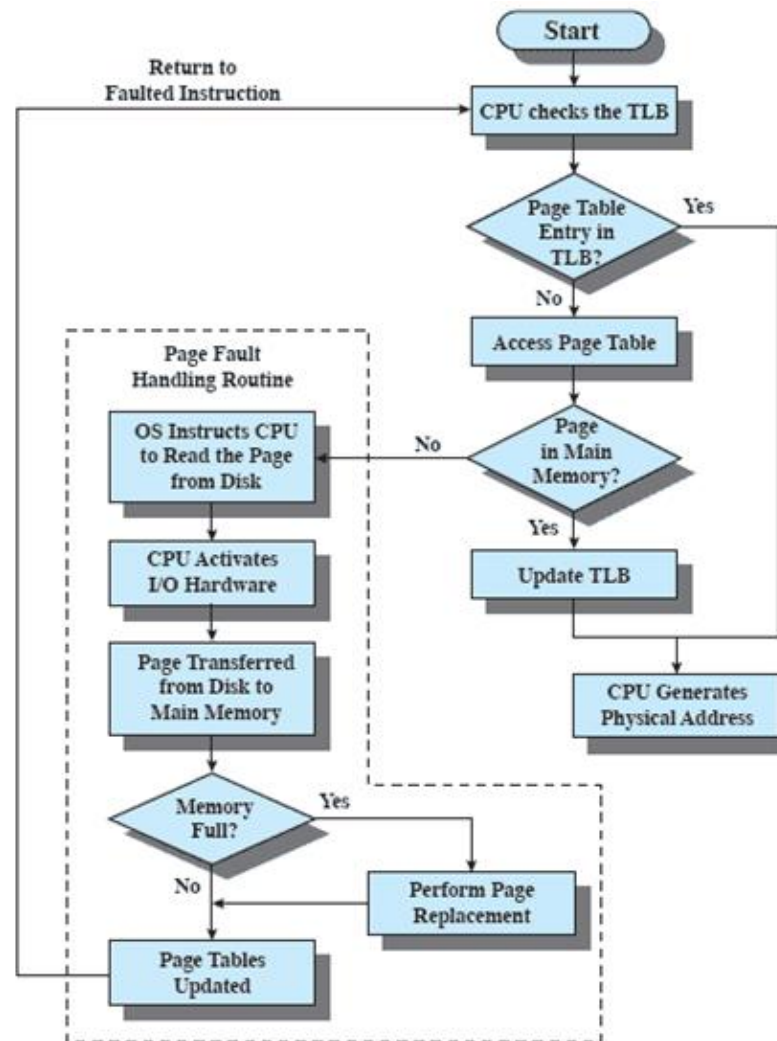


Figure 8.8 Operation of Paging and Translation Lookaside Buffer (TLB) [FURH87]

EXERCİTİU

Suppose

- virtual memory of 2^{32} bytes
- physical memory of 2^{24} bytes
- page size is 2^{10} (1 K) bytes

VPN		R	D	PPN
0		0	0	7
1		1	1	9
2		1	0	0
3		0	0	5
4		1	0	5
5		0	0	3
6		1	1	2
7		1	0	4
8		1	0	1

1. How many pages can be stored in physical memory at once?
2. How many entries are there in the page table?
3. How many bits are necessary per entry in the page table? (Assume each entry has PPN, resident bit, dirty bit)
4. How many pages does the page table require?
5. What's the largest fraction of VM that might be resident?
6. A portion of the page table is given to the left. What is the physical address for virtual address 0x1804?

EXERCİTİU

Suppose

- virtual memory of 2^{32} bytes
- physical memory of 2^{24} bytes
- page size is 2^{10} (1 K) bytes

VPN		R	D	PPN
0		0	0	7
1		1	1	9
2		1	0	0
3		0	0	5
4		1	0	5
5		0	0	3
6		1	1	2
7		1	0	4
8		1	0	1

1. How many pages can be stored in physical memory at once? $2^{24-10} = 2^{14}$
2. How many entries are there in the page table? 2^{22}
3. How many bits are necessary per entry in the page table? (Assume each entry has PPN, resident bit, dirty bit) 16
4. How many pages does the page table require? $2^{23} \text{ bytes} = 2^{13} \text{ pages}$
5. What's the largest fraction of VM that might be resident? $1/2^8$
6. A portion of the page table is given to the left. What is the physical address for virtual address 0x1804?

VPN=6 $\Rightarrow$ PPN=2 $\Rightarrow$ PA=0x804

TLB ȘI MEMORIA CACHE

○ Conceptul de cache

- Zonă de **stocare temporară**, unde informația **accesată frecvent** poate fi stocată pentru **acces rapid**
 - Data din cache este o copie a datei originale – care ar fi mult mai costisitor de accesat

○ TLB este un cache pentru translatare mai rapidă

- Eficientizează mecanismul de mapare din spațiul de adrese al procesului în spațiul de adrese al memoriei fizice
- Stochează mapări recente (similar cu intrările în tabela de pagini)
 - adresă virtuală : adresă fizică (plus biți de control)
- Căutare eficientă – Content-Addressable Memory (CAM)

○ Memoria CACHE propriu-zisă: pentru acces rapid la date

- Eficientizează accesul la datele / instrucțiunile din memoria fizică accesate recent
- Viteză mare de acces

LATENȚE LA ACCESUL DATELOR

- Latențe în accesul la
 - Cache
 - Memoria internă
 - Disc
 - etc.

Latency Numbers Every Programmer Should Know

0.5 ns	- CPU L1 dCACHE reference
1 ns	- speed-of-light (a photon) travel a 1 ft (30.5cm) distance
5 ns	- CPU L1 iCACHE Branch mispredict
7 ns	- CPU L2 CACHE reference
100 ns	- MUTEX lock/unlock
100 ns	- Main DDR MEMORY reference
10,000 ns	- Compress 1K bytes with Zippy PROCESS
20,000 ns	- Send 2K bytes over 1 Gbps NETWORK
250,000 ns	- Read 1 MB sequentially from MEMORY
500,000 ns	- Round trip within a same DataCenter
10,000,000 ns	- DISK seek
10,000,000 ns	- Read 1 MB sequentially from NETWORK
30,000,000 ns	- Read 1 MB sequentially from DISK
150,000,000 ns	- Send a NETWORK packet CA -> Netherlands

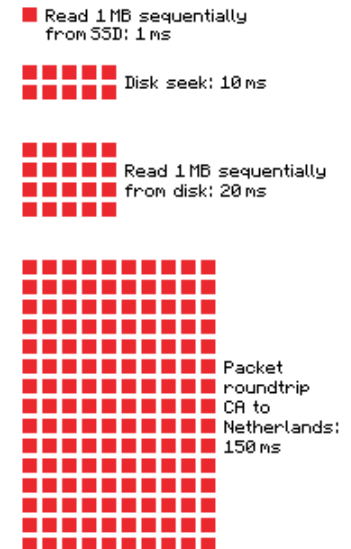
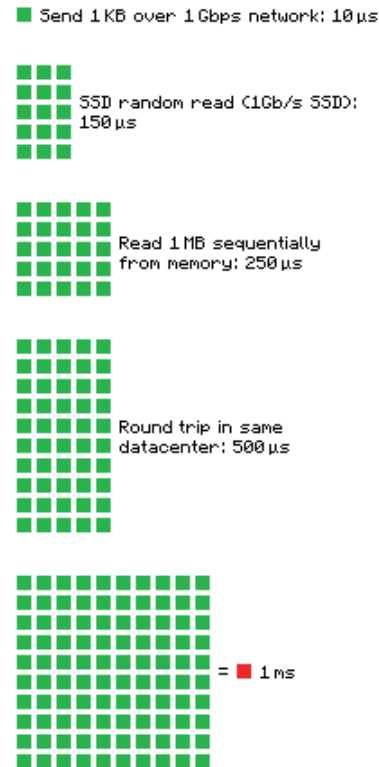
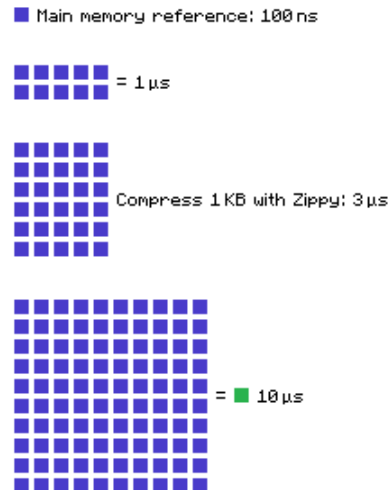
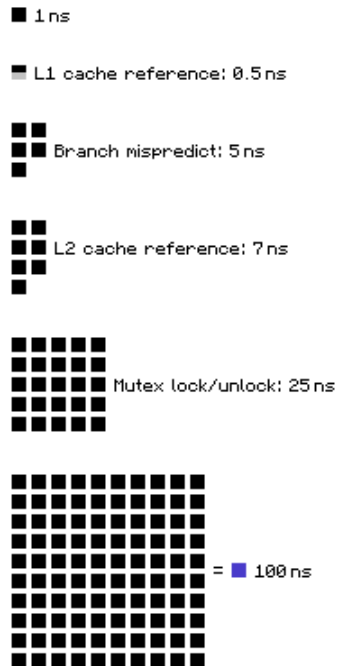
ms

us

ns

LATENȚE LA ACCESUL DATELOR

Latency Numbers Every Programmer Should Know



Source: <https://gist.github.com/2841832>

TLB ȘI MEMORIA CACHE

Stallings, pg. 353

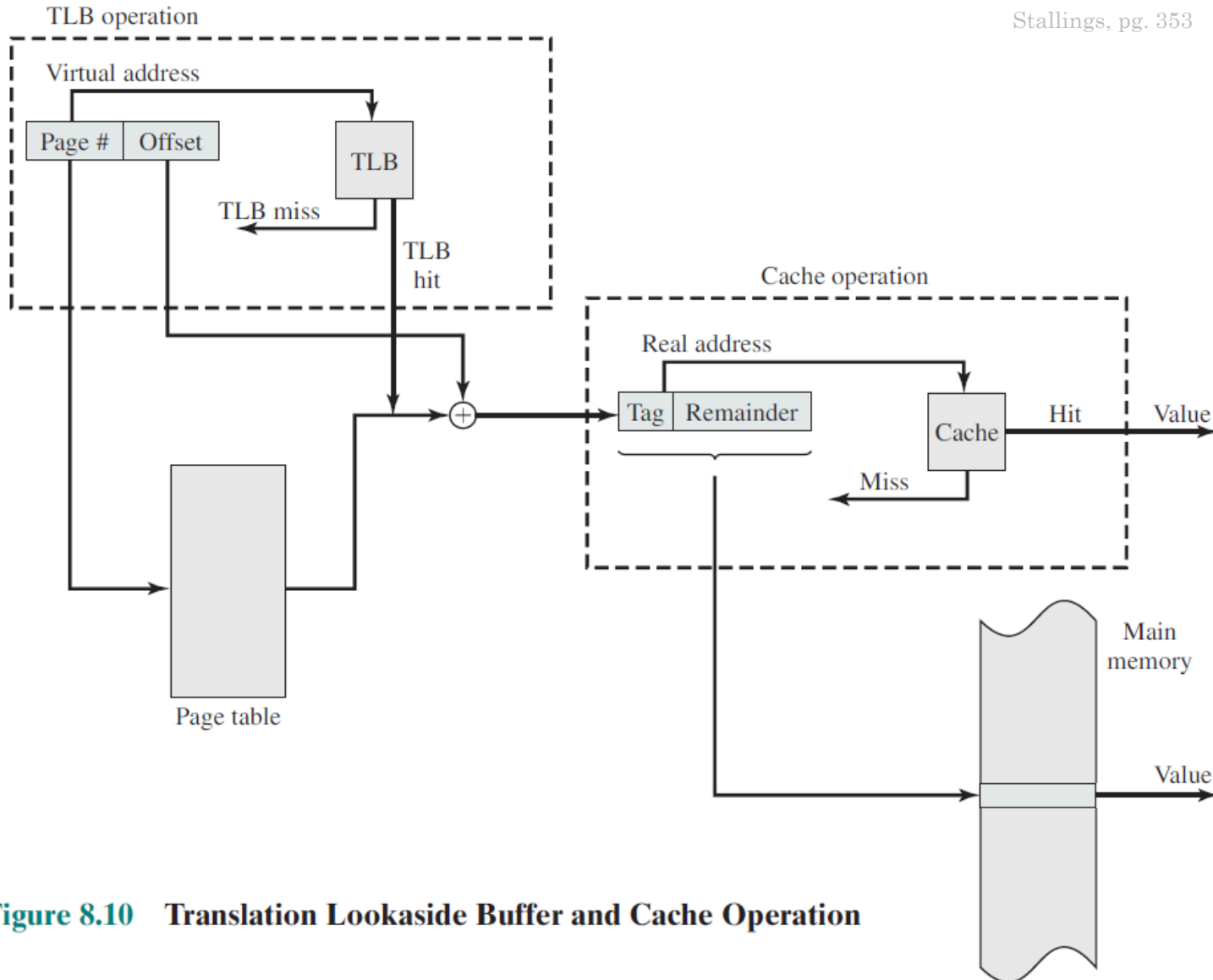
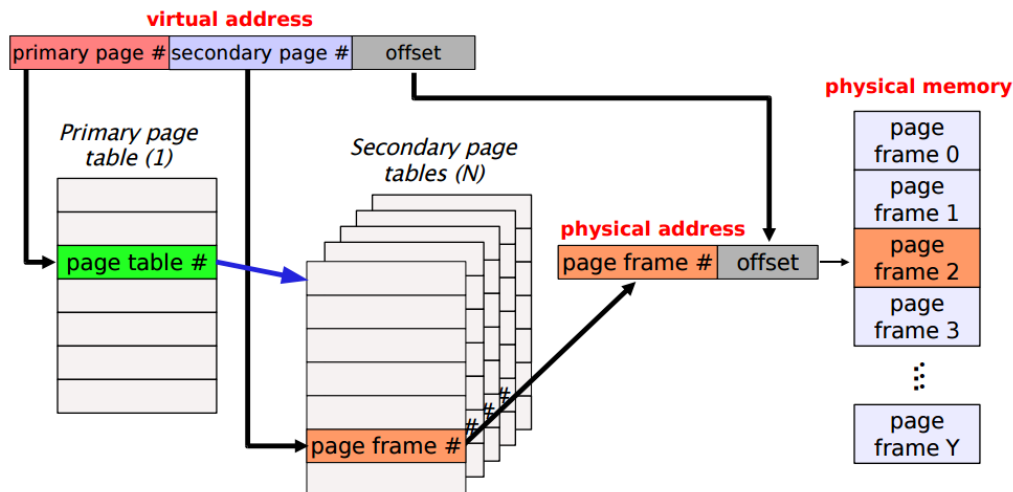


Figure 8.10 Translation Lookaside Buffer and Cache Operation

TABELE DE PAGINI IERARHICE

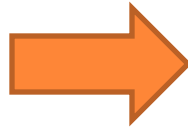
- **Motivație:** spațiul necesar stocării tabelului de pagini poate deveni f. mare
 - Ex. Pt. sp. de adrese virtual de 2 GB (2^{31}), pagina de 512 B (2^9) => tabela de pagini are 2^{22} intrări
- **Soluție**
 - tabela de pagini în memoria virtuală
 - Tabela de pagini este paginată
 - Încărcarea în memoria internă a unei părți a tablelei de pagini



TABELE DE PAGINI IERARHICE

- Ex. de tabelă ierarhică pe 2 niveluri

- Sp de adrese 4 GB (2^{32})
- Pagini de 4 KB (2^{12})
- Nr de pagini: 2^{20}
- Intrare in tabela: 4 B



- Tabela de pagini ocupă 4 MB (2^{22})
adică 2^{10} pagini
 - Se stochează pe disc
- Tabela rădăcină mapează tabela de pagini cu 2^{10} intrări a câte 4 B fiecare
 - Deci are 4 KB (2^{12})
 - Se păstrează în memoria internă

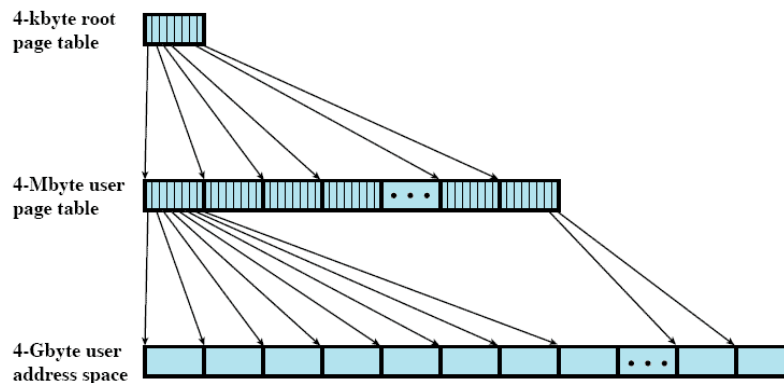


Figure 8.4 A Two-Level Hierarchical Page Table

TABELE DE PAGINI IERARHICE

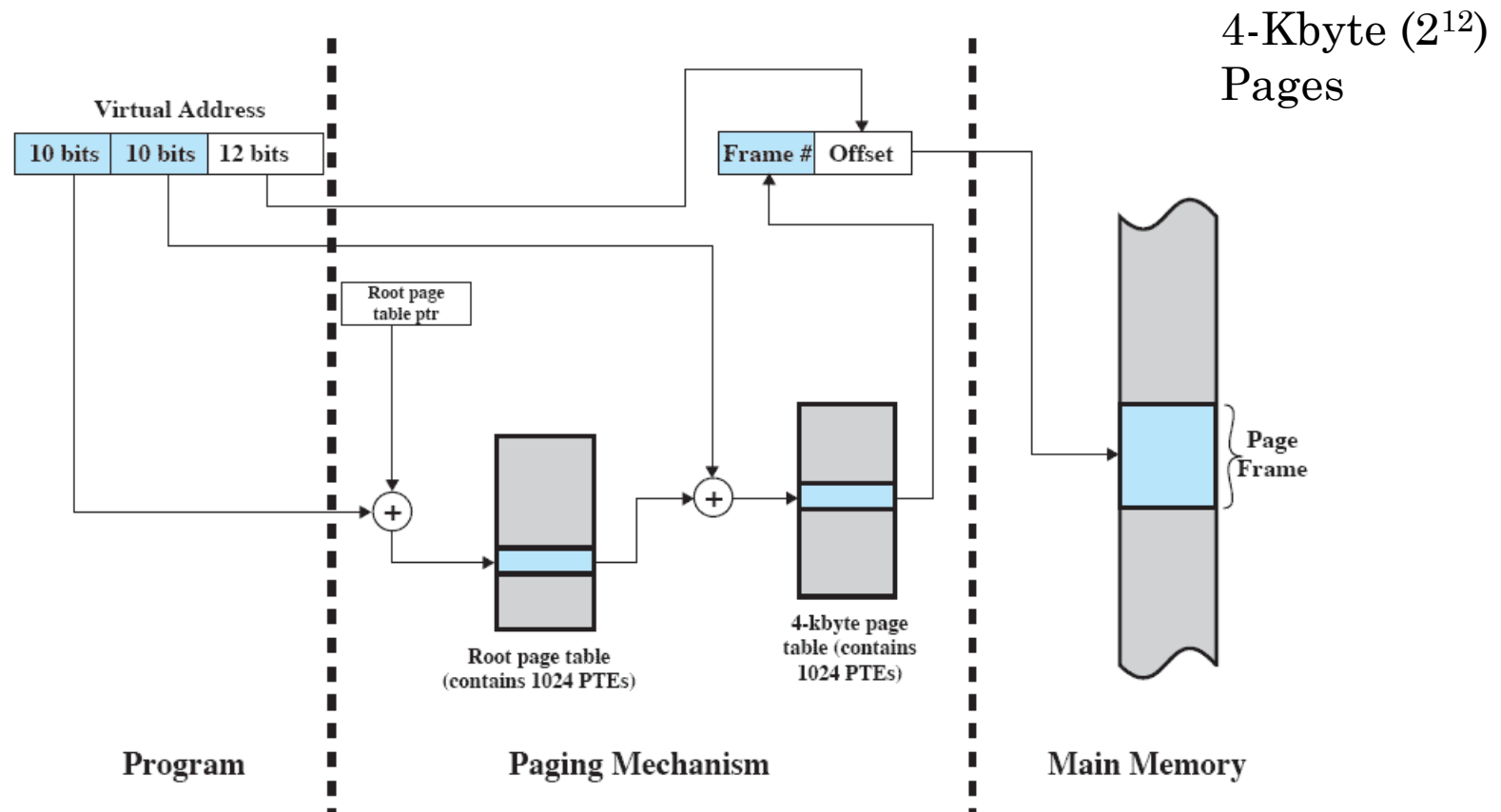
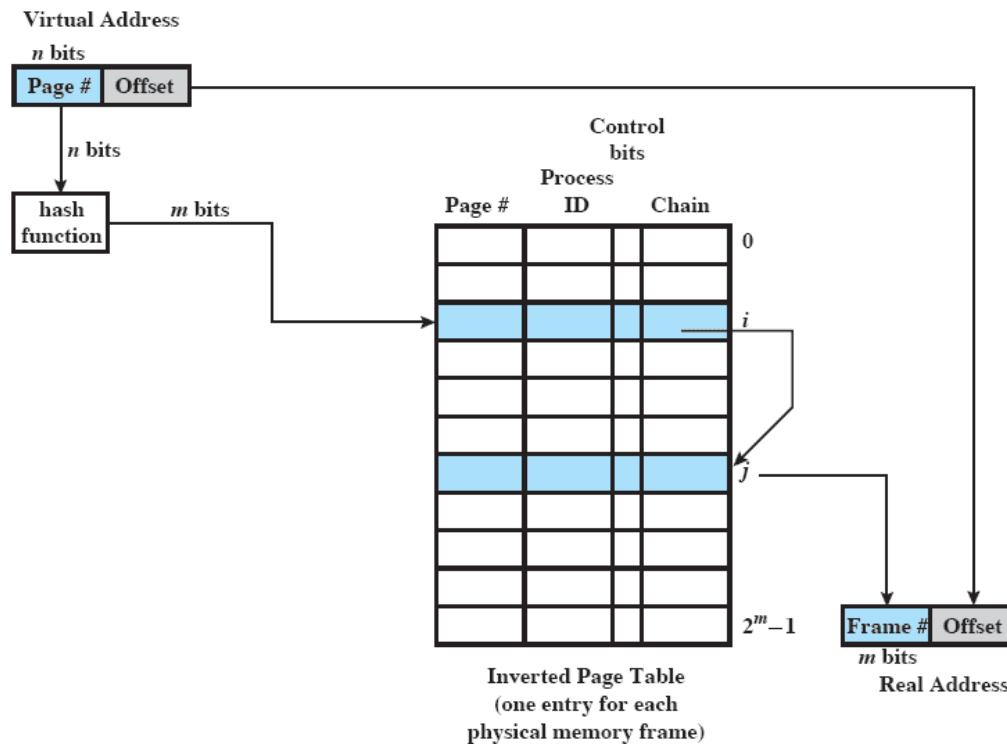


Figure 8.5 Address Translation in a Two-Level Paging System

TABELE DE PAGINI INVERSATE

- Tabelele de pagină prezentate până acum au un mare dezavantaj: dimensiunea tabelului este proporțională cu dimensiunea spațiului de adrese virtual
- Tabela de pagini indexată de numărul de **cadru de pagină**
- Tabelă de **dimensiune fixă**
 - Nu depinde de numărul de procese sau numărul de pagini virtuale



O singură tabelă per sistem
Trebuie să păstreze informații despre proces

Figure 8.6 Inverted Page Table Structure

DIMENSIUNEA UNEI PAGINI

- Paginarea prezintă problema **fragmentării interne**
 - Ultima pagină poate fi doar parțial folosită
- Dimensiune **mică**
 - Avantaj
 - Reduce fragmentarea internă
 - Dezavantaj
 - Mai multe pagini / proces
 - Tabele de pagini mai mari
 - Parțial pe disc -> număr dublu de page fault-uri
- Dimensiune **mare**
 - Avantaj
 - Transfer de date mai eficient (între disc și memoria internă)
 - Dezavantaj
 - Grad mai mare de fragmentare internă
- Decizia este rezultatul unui compromis (4KB – 4MB - 16MB – 256MB)
- Cele mai multe sisteme suportă o **singură dimensiune** de pagină
- Tot mai multe sisteme oferă suport pentru **multiple dimensiuni** de pagină
 - Regiuni mari contigue (ex. codul) pot fi mapate folosind un număr redus de pagini mari,
 - Stiva threadurilor mapată într-un număr mare de pagini mai mici

SEGMENTARE

FOLOSIND MEMORIE VIRTUALĂ

SEGMENTARE

○ Viziune diferită asupra memoriei

- Unitate logică
 - Programul divizat în **segmente cu funcționalități specifice** și de dimensiuni variate
- Tabela de segmente
 - O intrare conține
 - Adresa de început a segmentului în memoria fizică
 - Dimensiunea segmentului
 - Biți de control
 - Bit de prezență în memoria fizică
 - Bit de modificare

○ Avantaje

- Simplifică gestiunea **structurilor** de date **în creștere**
- Permite modificarea și recompilarea independentă a programelor
 - Fără a fi necesară reeditarea legăturilor și reîncărcarea întregului set de programe
- Permite **partajarea** ușoară a datelor
 - Plasarea în segmente partajate
- Permite **protecția** ușoară a datelor
 - Asignarea unor drepturi de acces la segmente

SEGMENTARE

- Translația adresei logice la adresa fizică

Stallings, pg. 357

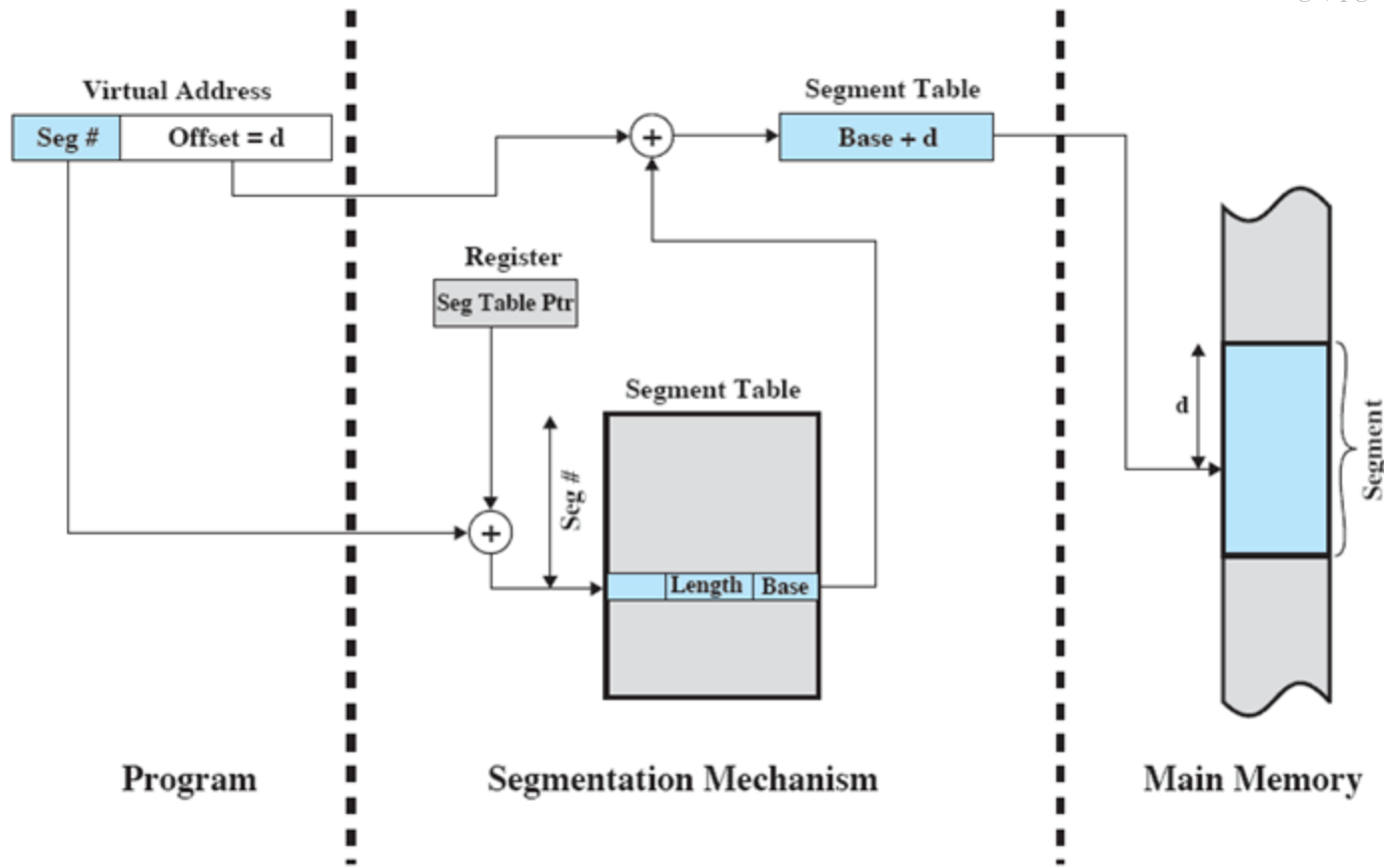


Figure 8.12 Address Translation in a Segmentation System

CARACTERISTICILE PAGINĂRII ȘI SEGMENTĂRII

Simple Paging	Virtual Memory Paging	Simple Segmentation	Virtual Memory Segmentation
Main memory partitioned into small fixed-size chunks called frames	Main memory partitioned into small fixed-size chunks called frames	Main memory not partitioned	Main memory not partitioned
Program broken into pages by the compiler or memory management system	Program broken into pages by the compiler or memory management system	Program segments specified by the programmer to the compiler (i.e., the decision is made by the programmer)	Program segments specified by the programmer to the compiler (i.e., the decision is made by the programmer)
Internal fragmentation within frames	Internal fragmentation within frames	No internal fragmentation	No internal fragmentation
No external fragmentation	No external fragmentation	External fragmentation	External fragmentation
Operating system must maintain a page table for each process showing which frame each page occupies	Operating system must maintain a page table for each process showing which frame each page occupies	Operating system must maintain a segment table for each process showing the load address and length of each segment	Operating system must maintain a segment table for each process showing the load address and length of each segment
Operating system must maintain a free frame list	Operating system must maintain a free frame list	Operating system must maintain a list of free holes in main memory	Operating system must maintain a list of free holes in main memory
Processor uses page number, offset to calculate absolute address	Processor uses page number, offset to calculate absolute address	Processor uses segment number, offset to calculate absolute address	Processor uses segment number, offset to calculate absolute address
All the pages of a process must be in main memory for process to run, unless overlays are used	Not all pages of a process need be in main memory frames for the process to run. Pages may be read in as needed	All the segments of a process must be in main memory for process to run, unless overlays are used	Not all segments of a process need be in main memory for the process to run. Segments may be read in as needed
	Reading a page into main memory may require writing a page out to disk		Reading a segment into main memory may require writing one or more segments out to disk

SEGMENTARE CU PAGINARE

SEGMENTARE CU PAGINARE

○ Abordare combinată

- Spațiul de adrese al procesului este divizat în **segmente de dimensiuni variate**
 - Aspect vizibil programatorului
- Fiecare segment este **divizat în pagini** de dimensiuni fixe
 - Aspect transparent programatorului
- **Combină beneficiile** specifice paginării și segmentării
 - Beneficii specifice paginării
 - Gestiune transparentă programatorului
 - Eliminarea fragmentării externe
 - Utilizarea eficientă a memoriei
 - Pagini de dimensiuni fixe (dimensiuni egale)
 - Permite dezvoltarea algoritmilor sofisticati care exploatează comportamentul programelor
 - Beneficiile specifice segmentării
 - Vizibil programatorului
 - Gestiunea structurilor de date în creștere
 - Suport pentru modularitate
 - Suport pentru partajare și protecție

SEGMENTARE CU PAGINARE

- Abordare combinată
 - Spațiul de adrese al procesului este divizat în segmente de dimensiuni variate
 - Aspect vizibil programatorului
 - Fiecare segment este divizat în pagini de dimensiuni fixe
 - Aspect transparent programatorului
 - Combină beneficiile specifice paginării și segmentării
- Adresa logică
 - număr de segment, număr de pagină, deplasament
- Formatul intrărilor în tabela paginilor și tabela segmentelor

Virtual Address



Stallings, pg. 346

Segment Table Entry



Page Table Entry



P = present bit
M = Modified bit

(c) Combined segmentation and paging

SEGMENTARE CU PAGINARE

Stallings, pg. 358

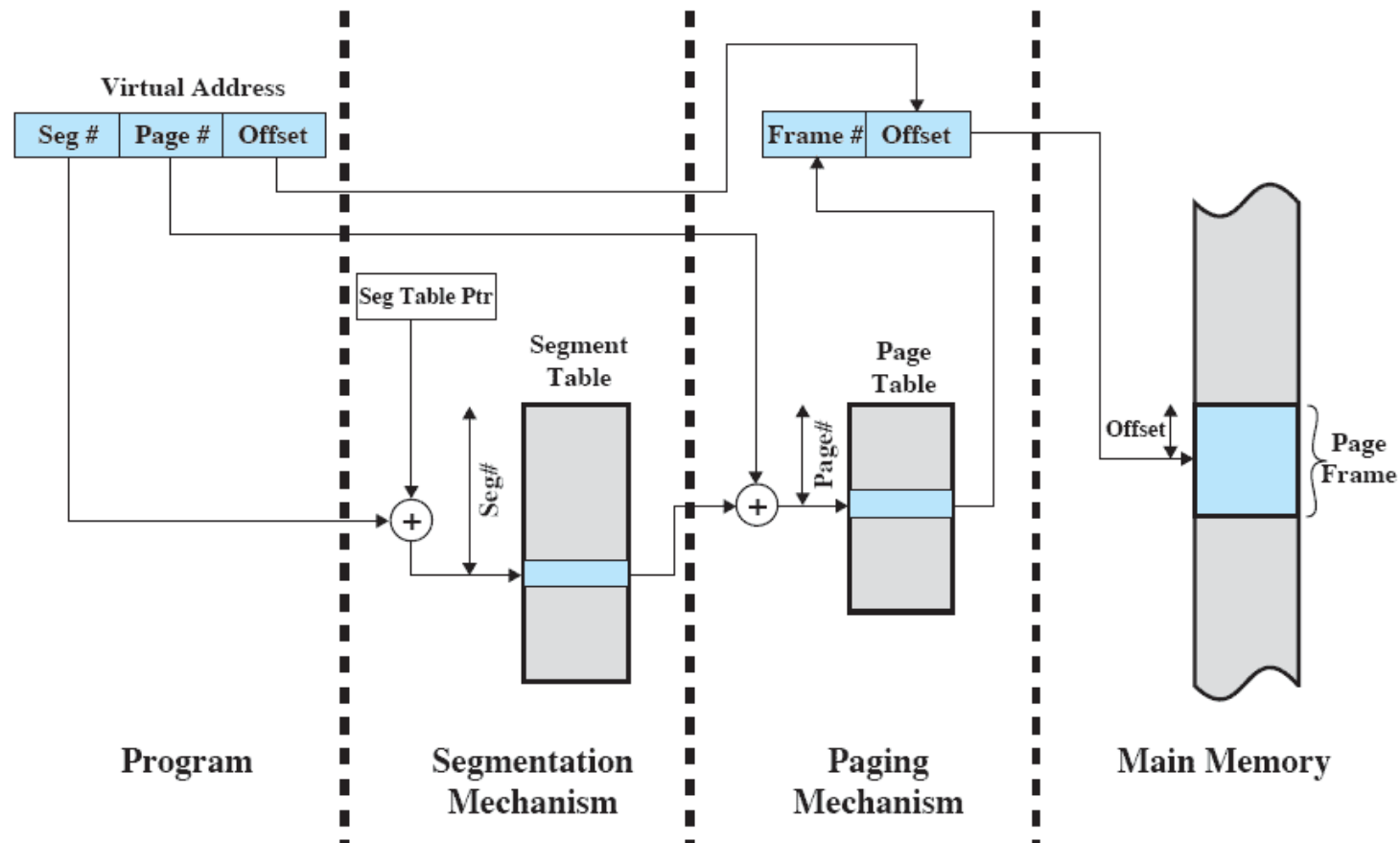


Figure 8.13 Address Translation in a Segmentation/Paging System

PROTECȚIA ȘI PARTAJAREA

- Decurg din caracteristicile segmentării
 - Permite **partajarea** ușoară a datelor
 - Plasarea în segmente partajate
 - Controlul accesului
 - Permite **protecția** ușoară a datelor
 - Asignarea unor drepturi de acces la segmente
 - Fiecare intrare are o pereche de adrese de început și dimensiune
- Mecanisme de partajare și protecție sunt disponibile și în sisteme cu paginare, însă structura de pagini ale programelor nu este vizibilă programatorului
 - Specificarea acestor mecanisme nu este atât de ușoară (și naturală) ca și în cazul segmentării

COPY-ON-WRITE

COPY-ON-WRITE

○ Crearea unui proces nou

○ Context

- Procesul fiu este **copia părintelui**
- Operație costisitoare
 - Crearea unui spațiu de adrese noi
 - Copierea procesului părinte – pagină cu pagină
- De multe ori după fork fiul apelează **exec**
 - Copierea a fost inutilă

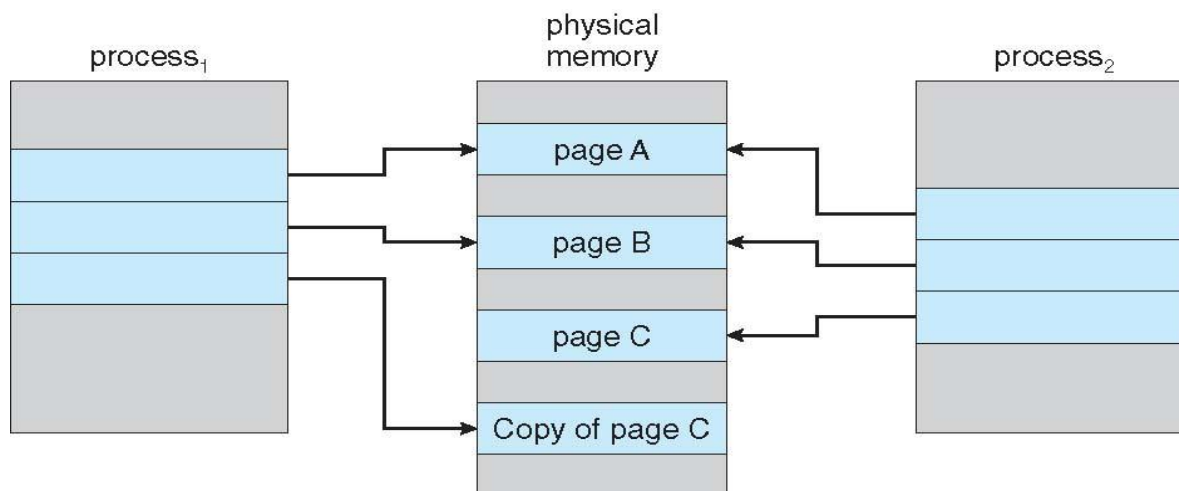
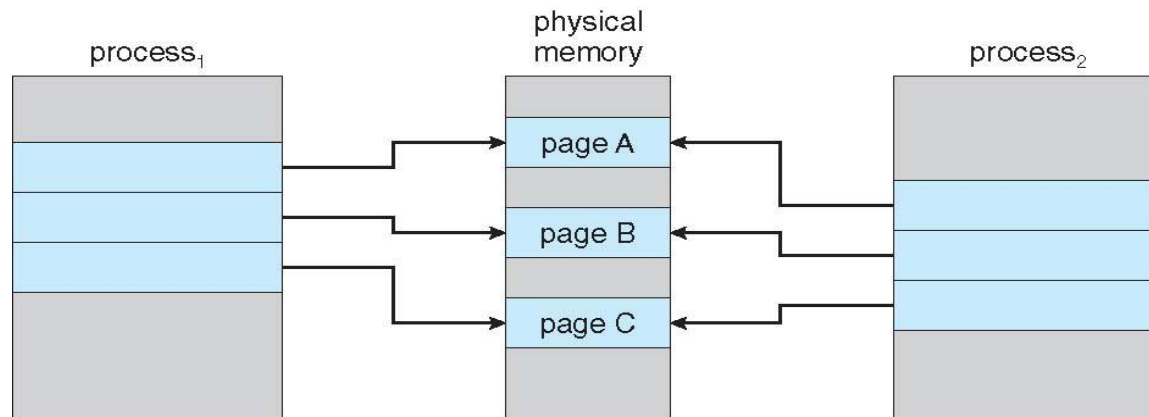
○ Soluție

• **Copy-on-write**

- Procesul părinte și fiu partajează paginile din memorie în citire
- Dacă unul din procese modifică (scrie) o pagină
 - Page-fault -> se creează o copie a paginii accesibilă doar procesului care a cerut modificarea
 - Pagina originală va rămâne nemodificată și doar celălalt proces va avea acces la ea

COPY-ON-WRITE

Silberschatz, pg. 368



DECIZII DE PROIECTARE MMU

GESTIUNEA MEMORIEI

- Decizii de proiectare a MMU
 - Integrarea / sau nu a tehnicilor care folosesc **memorie virtuală**
 - Folosirea **paginării**, folosirea **segmentării** sau a **ambelor**
 - **Alegerea algoritmilor** de gestiune a celorlalte aspecte
- Primele două depind de platforma hw
 - Paginarea și segmentarea nu sunt practice fără suport hardware
 - Translatarea adresei și celelalte operații trebuie implementate în hw pt eficiență
- Cele mai multe sisteme furnizează suport pt memoria virtuală
- Sistemele cu segmentare pură sunt tot mai rare
- La segmentarea cu paginare dificultatea este dată de gestiunea paginării
 - În continuare sunt prezentate aspecte legate de paginare
- Cea de a treia decizie depinde de SO – pe acestea le discutăm în continuare

MEMORIA VIRTUALĂ - POLITICI

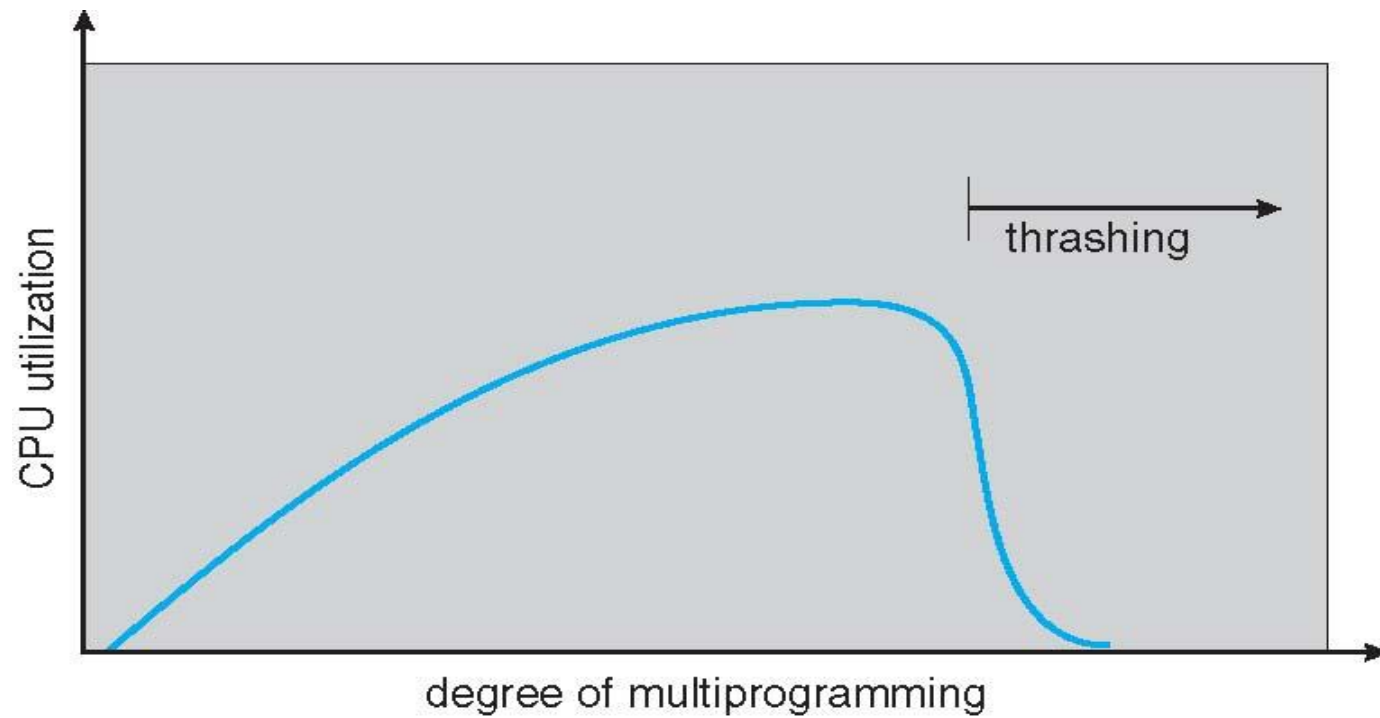
- Scop
 - Maximizarea performanței
 - Minimizarea page fault-urilor
 - Page-fault: excepție cauzată de absența din memoria fizică a unei pagini referite
 - Minimizarea încărcării software cauzate de page-fault (înlocuire, schimbare de context, etc.)
- Politica de **încărcare** a unei pagini în memorie
 - Paginare la cerere
 - Pre-paginare
- Politica de **plasare** a procesului în memorie
- Algoritmi de **înlocuire** a paginilor
- Politica de gestiune a **setului rezident**
- Politica de **curățare** - scrierea paginilor încărcate înapoi pe suport
 - Curățare la cerere
 - Pre-curățare

THRASHING

THRASHING — PAGINARE EXCESIVĂ

- Stare în care sistemul petrece mare parte din timp efectuând swapping asupra părților din proces
 - Rămâne foarte puțin timp pentru procesarea efectivă
- În sisteme care au un număr de cadre de pagină relativ mic, rata de page-fault-uri este foarte mare
 - Înlocuirea unei pagini care va fi imediat după aceea referită
- Gradul de utilizare a procesorului scade
- SO încearcă să crească gradul de multiprogramare
 - Adaugă noi procese – încarcă în memorie
 - Îngreunează și mai mult gestiunea memoriei

THRASHING

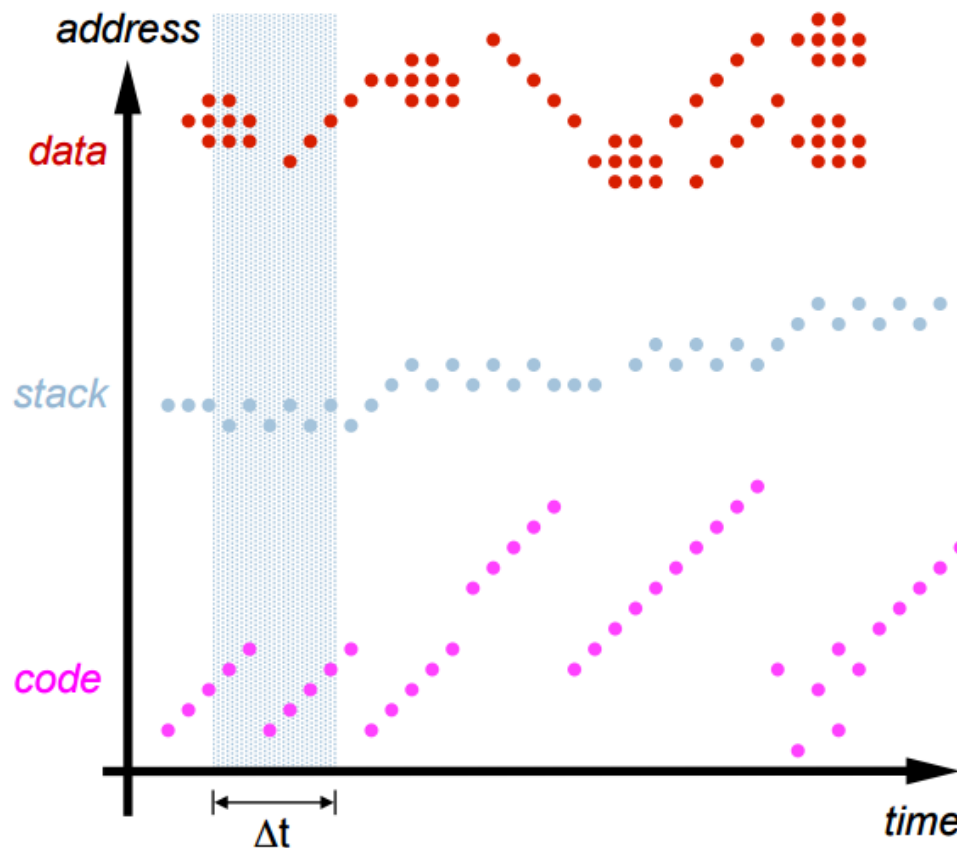


PRINCIPIUL LOCALITĂȚII

- Sistemul încearcă să ghicească zonele cu probabilitate mare de accesare în viitorul apropiat
 - Bazat pe istoria recentă
 - Pentru a evita thrashing-ul
- Caracteristică
 - **Referințele** la instrucțiuni și date **tind să se grupeze**
 - Doar a parte mică din proces este necesar într-o perioadă scurtă
- Tipuri de localitate
 - **Localitate temporală**
 - Reutilizarea frecventă a unor date într-o perioadă scurtă de timp
 - **Localitate spațială**
 - Datele referite sunt grupate în aceeași zonă de memorie
- Introduce predictibilitate

PRINCIPIUL LOCALITĂȚII

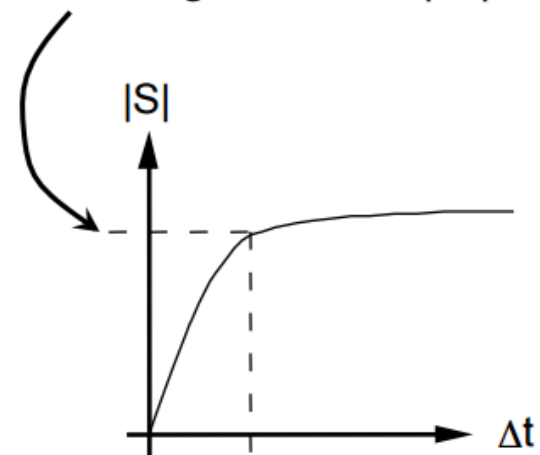
- Șabloane de referință a memoriei



S is the set of locations accessed during Δt .

Working set: a set S which changes slowly wrt access time.

Working set size, $|S|$



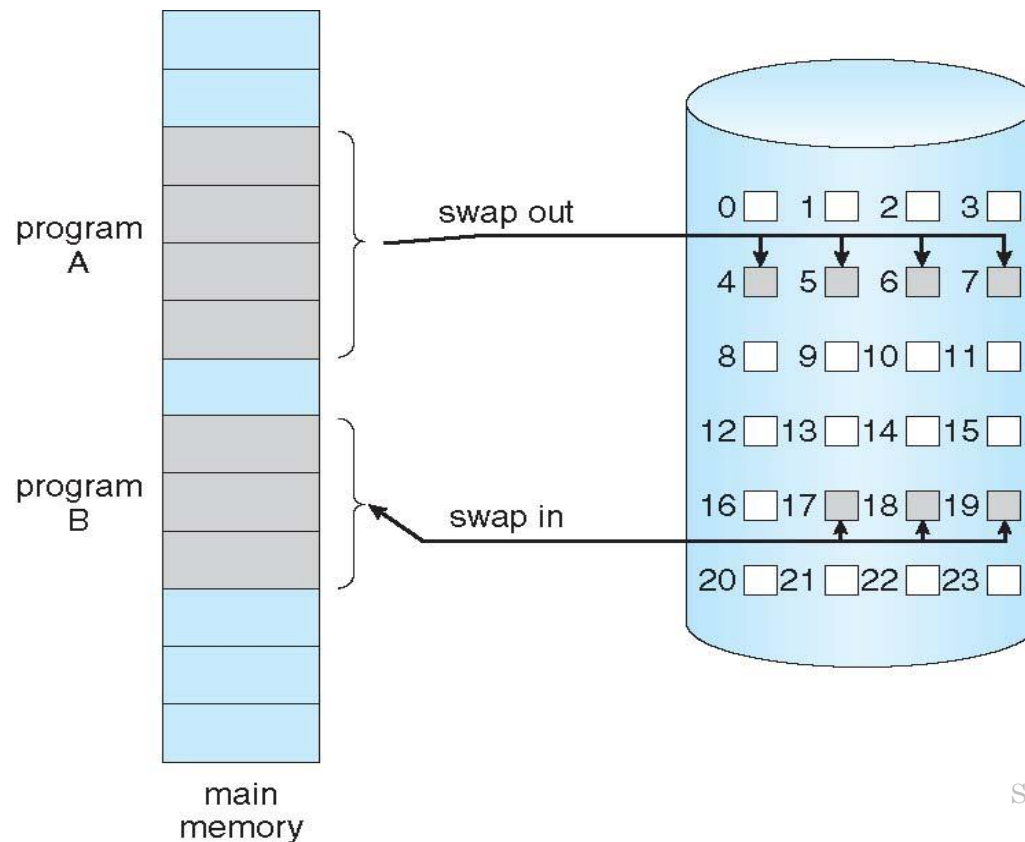
PAGINARE LA CERERE

PAGINARE LA CERERE

- O pagină este încărcată în memorie **doar când este referită** o locație din aceea pagină
 - Cerere pt pagină
 - Referință invalidă -> eșuează
 - Nu este în memorie -> încărcă
- Multe page fault-uri la începutul execuției procesului
 - Nici o pagină nu este încărcată
- Principiul localității
 - Sugerează că
 - referințele viitoare au probabilitate mare să acceseze
 - pagini accesate precedent sau
 - locații din apropierea acestora
 - deci care au probabilitate mare să fie deja în memorie
- **Page fault-urile scad în timp** la un nivel foarte mic

PAGINARE LA CERERE

- Trasferul memoriei paginate între memoria internă și disc – în zone contigue
 - Lucrează cu un subset de pagini
 - La swapping întregul proces este scos din memoria internă



PAGINARE LA CERERE

- Intrările în tabela de pagini includ
 - P – bit care indică **prezența** acelei pagini în memoria internă
 - Dacă bitul este setat atunci intrarea include și adresa cadrului de pagină din memoria fizică
 - Valori
 - v - valid – în memorie
 - i - invalid – nu este în memorie
 - Inițial toate valorile sunt *i*
 - La translatarea adresei
 - dacă $P = i$
 - page fault

PAGINARE LA CERERE

- Tabele de pagini
 - Situație când nu sunt toate paginile în memorie

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

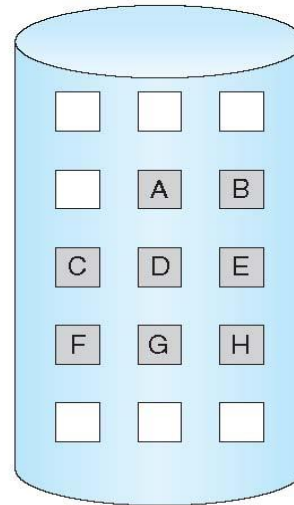
logical
memory

valid-invalid bit		
frame		
0	4	v
1		i
2	6	v
3		i
4		i
5	9	v
6		i
7		i

page table

0	
1	
2	
3	
4	A
5	
6	C
7	
8	
9	F
10	
11	
12	
13	
14	
15	

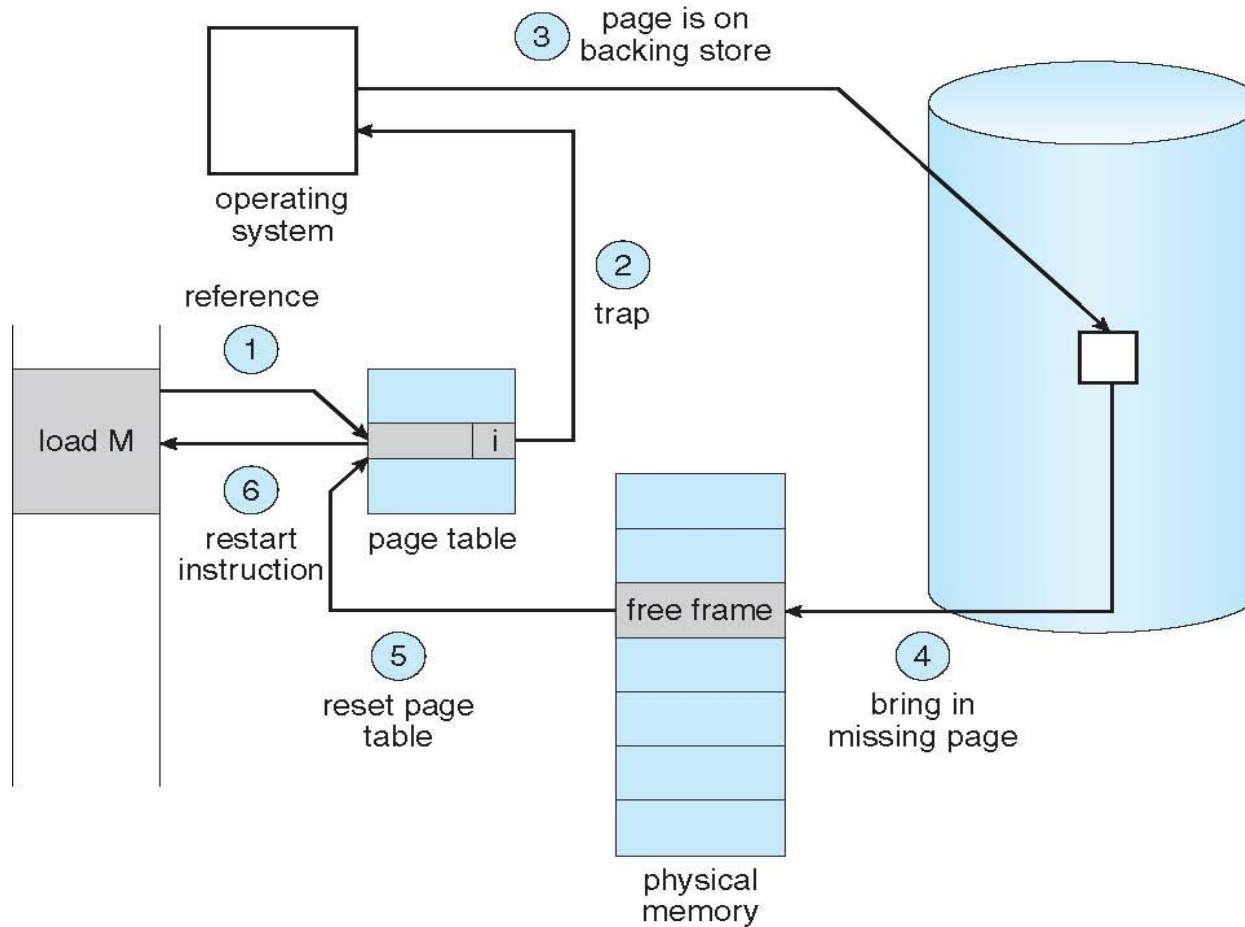
physical memory



Silberschatz, pg. 362

PAGINARE LA CERERE

○ Tratarea întreruperii page fault



PAGINARE LA CERERE – PERFORMANȚA

○ Rata page fault-urilor

- $p = 0$
 - Toate paginile referinte sunt în memorie
- $p = 1$
 - Nici o pagină referită nu este în memorie

○ Timp de acces efectiv (EAT)

- $EAT = (1 - p) * \text{access_time} + p * \text{service_time}$
 - service_time
 - Timpul necesar tratării page fault-ului

• Exemplu

- Acces la memorie (access_time): 200 nanosec
- Deservirea page fault-ului (service_time): 8 milisec
- $EAT = (1 - p) \times 200 + p \times 8000000$
 $= 200 + p \times 7999800$

Dacă un acces din 1000 provoacă page fault: $EAT = 8.2 \mu\text{sec}$

- Încetinire cu un factor de 40

- Dacă vrem o încetinire cu maxim 10%
 - < 1 acces din 400 000 provoacă page fault

PRE-PAGINARE

PRE-PAGINARE

- Sunt încărcate și alte pagini în afara paginii cerute
 - La referirea unei locații dintr-o pagină care nu este încărcată în memorie
- Exploatează caracteristicile celor mai multe dispozitive de stocare secundare
 - Dacă paginile sunt stocate în zone contigue este mai eficient să se aducă mai multe pagini în memorie
- Poate fi ineficient
 - dacă nu se face referire la paginile încărcate adițional
- Poate fi apelat
 - La pornirea în execuție a procesului sau
 - La fiecare page fault
- Nu confundați cu swapping-ul

PLASARE

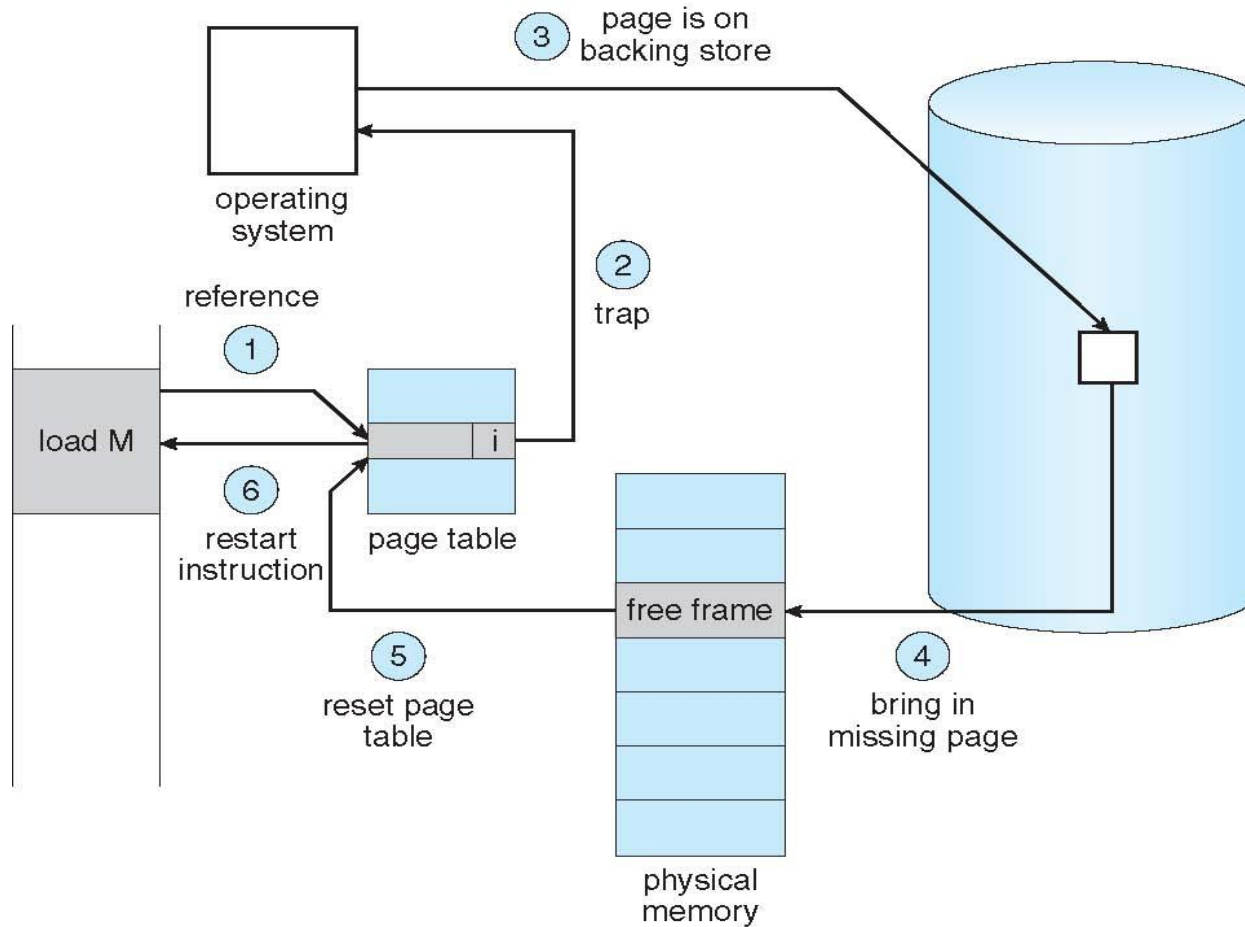
POLITICA DE PLASARE

- **Determină locul din memoria fizică** unde va fi încărcat procesul (sau o parte din proces)
- Are importanță în
 - Sisteme bazate exclusiv pe segmentare (segmentare pură)
 - Best-fit, first-fit, next-fit, worst-fit, etc.
 - Sisteme multiprocesor cu NUMA (NonUniform Memory Access)
 - Memoria partajată este distribuită
 - Poate fi accesată de oricare dintre procesoare
 - Timpul de acces depinde de distanța dintre procesor și memoria accesată
 - Performanța depinde în mare măsură de plasarea procesului aproape de procesorul pe care va fi rulat
- Irelevant la paginare pură sau segmentare cu paginare
 - Hw de traducere a adreselor funcționează la fel de eficient pt orice combinație pagină – cadru de pagină

ALGORITMI DE ÎNLOCUIRE A PAGINILOR

PAGINARE LA CERERE

○ Tratarea întreruperii page fault



ALGORITMI DE ÎNLOCUIRE A PAGINILOR

○ Selectarea paginii din memorie care va fi înlocuită

- În momentul în care trebuie încărcată o pagină nouă și
- Nu mai sunt cadre de pagină libere
- La eliminare, pagina selectată trebuie scrisă înapoi pe suport
 - Bit **M** – numit și dirty bit
 - Reduce încărcarea dată de transferul paginilor
 - Doar cele modificate sunt scrise pe disc

○ Scop

- Selectarea unei pagini care are probabilitate mică să fie referită în viitorul apropiat

○ Observații

- Permite folosirea unui spațiu de adrese virtual mult mai mare decât memoria fizică
 - Separare clară între memoria logică și memoria fizică
- Cu cât este mai complicat / sofisticat algoritmul folosit cu atât este mai mare încărcarea suplimentară pentru implementarea acestuia

ALGORITMI DE ÎNLOCUIRE A PAGINILOR

○ Pași necesari

1. Găsirea locației de pe disc a paginii cerute
2. Găsirea unui cadru de pagină din memoria internă
 1. Dacă există cadru liber – acesta este folosit
 2. Dacă nu există cadru liber - folosirea unui algoritm de înlocuire
 1. Selectarea unei pagini victimă
 2. Scrierea paginii victimă înapoi pe disc (dacă a fost modificată)
3. Încărcarea paginii noi în cadrul de pagină eliberat
 1. Se actualizează tabela paginilor și tabela cadrelor
4. Continuarea prin restartarea instrucțiunii care a provocat întreruperea - page fault-ul

○ EAT scade – două transferuri / page fault

ALGORITMI DE ÎNLOCUIRE A PAGINILOR

○ Cadre de pagină **blocate**

- Dacă un cadru de pagină este blocat, atunci pagina conținută de cadru nu poate fi înlocuită
 - Ex. Kernelul SO și structurile de control cheie pentru gestiunea resurselor sunt stocate în cadre blocate
- Implementat prin asocierea unui bit de blocare cu fiecare cadru

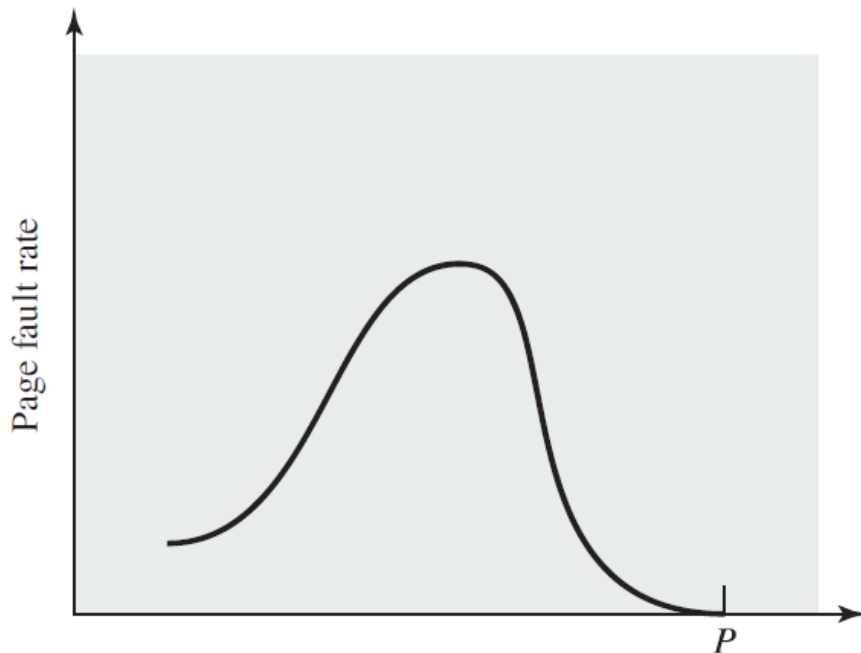
○ Algoritmi de **alocare a cadrelor**

- Determină numărul de cadre alocate unui proces
- Determină cadrele în care paginile pot fi înlocuite

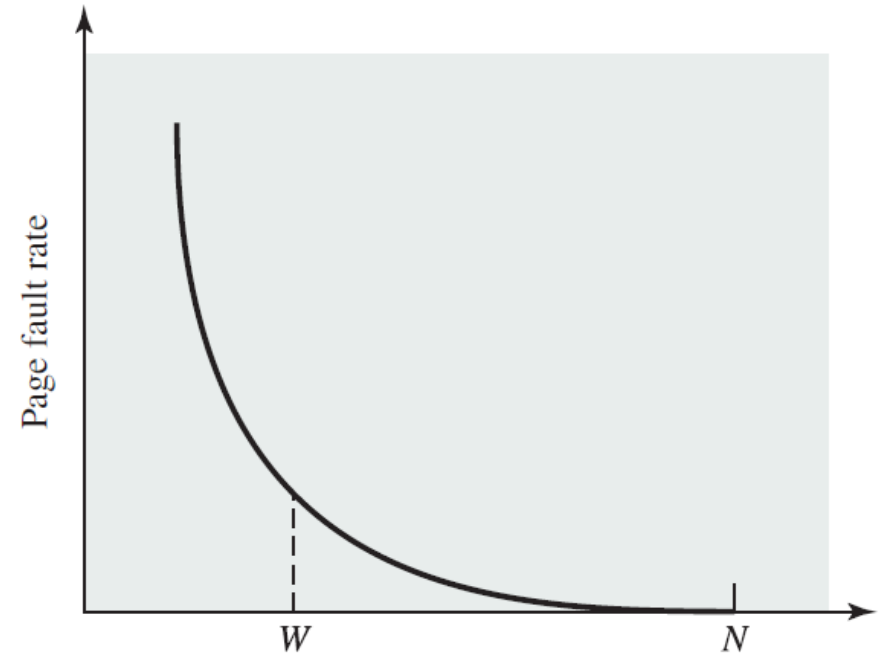
○ Algoritmi de **înlocuire a paginilor**

- Scopul urmărit este **minimizarea ratei de page fault-uri**
 - La primul acces și la reacesări

RATA DE PAGE FAULT-URI VS. NUMĂRUL DE CADRE



(a) Page size



(b) Number of page frames allocated

P = size of entire process

W = working set size

N = total number of pages in process

Figure 8.11 Typical Paging Behavior of a Program

ALGORITMI DE ÎNLOCUIRE A PAGINILOR

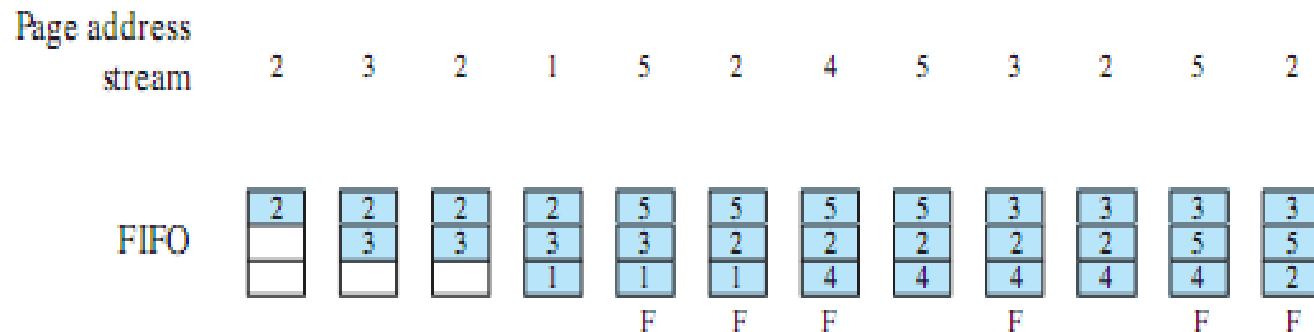
- FIFO
 - Optimal
 - LRU
 - Clock
-
- Comparăm performanța prin trasarea execuției pe un șir prestabilit de pagini referite
 - Paginile vor purta doar număr – nu adrese efective
 - Numărăm page fault-urile
-
- Primul șir
 - 2,3,2,1,5,2,4,5,3,2,5,2
 - Al doilea șir
 - 1,0,5,3,4,0,4,7,3,5,6,0,4,7,3,4,5

ALGORITMI DE ÎNLOCUIRE A PAGINILOR

○ FIFO - First-In First-Out

- Paginile sunt scoase în ordinea în care au fost încărcate în memorie
- Implementare simplă

Stallings, pg. 364



F = page fault occurring after the frame allocation is initially filled

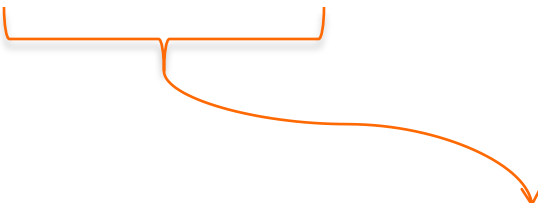
Figure 8.15 Behavior of Four Page Replacement Algorithms

6 page fault-uri

FIFO - FIRST-IN FIRST-OUT

○ Exemplu 2

Reference string	1	0	5	3	4	0	4	7	3	5	6	0	4	7	3	4	5
Physical frame 0	1	0	5	3	4	4	4	7	7	7	6	0	0	0	3	4	5
Physical frame 1		1	0	5	3	3	3	4	4	4	7	6	6	6	0	3	4
Physical frame 2			1	0	5	5	5	3	3	3	4	7	7	7	6	0	3
Physical frame 3				1	0	0	0	5	5	5	3	4	4	4	7	6	0
Page faults	F	F	F	F	F			F			F	F			F	F	F
Victim frames					1			0			5	3			4	7	6



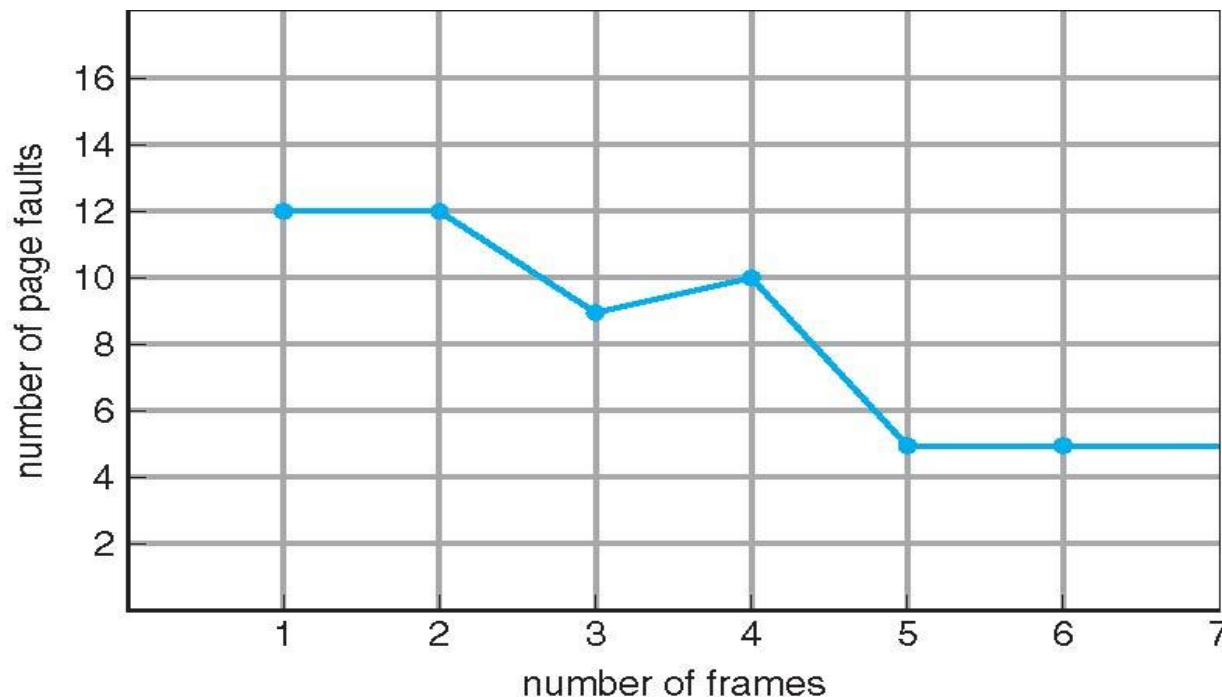
În această trasare se consideră și page fault-urile care apar la încărcarea primelor pagini

FIFO - FIRST-IN FIRST-OUT

○ Anomalia lui Bélády

- Adăugarea mai multor cadre de pagină poate duce la o rată mai mare de page fault-uri

Silberschatz, pg. 375



FIFO - FIRST-IN FIRST-OUT

○ Anomalia lui Bélády

- Adăugarea mai multor cadre de pagină poate duce la o rată mai mare de page fault-uri

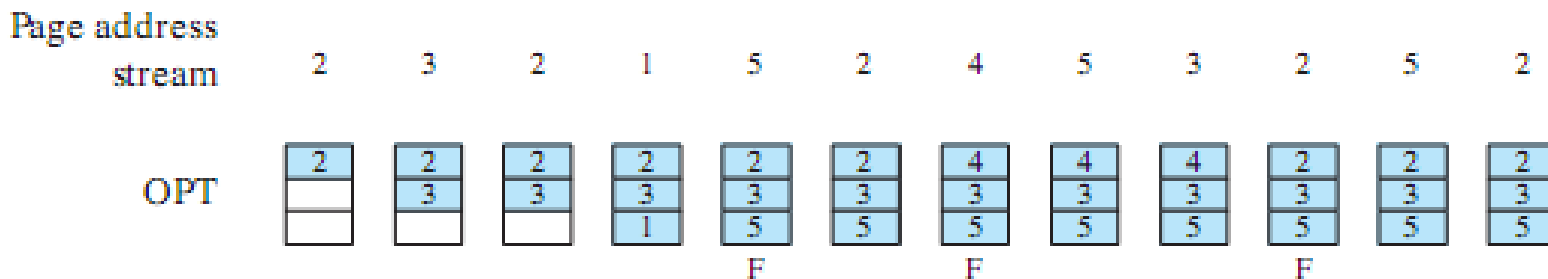
1	2	3	4	1	2	5	1	2	3	4	5
1	2	3	4	1	2	5	5	5	3	4	4
	1	2	3	4	1	2	2	2	5	3	3
		1	2	3	4	1	1	1	2	5	5

1	2	3	4	1	2	5	1	2	3	4	5
1	2	3	4	4	4	5	1	2	3	4	5
	1	2	3	3	3	4	5	1	2	3	4
		1	2	2	2	3	4	5	1	2	3
			1	1	1	2	3	4	5	1	2

ALGORITMUL OPTIMAL (AL LUI BÉLÁDY)

- Înlocuiește pagina care nu va fi folosită pentru cea mai lungă perioadă de timp
 - Pagina victimă este cea care este referintă după cea mai lungă perioadă de timp
- Problema - nu putem prezice viitorul

Stallings, pg. 364



F = page fault occurring after the frame allocation is initially filled

Figure 8.15 Behavior of Four Page Replacement Algorithms

ALGORITMUL OPTIMAL

○ Exemplu 2

Reference string	1	0	5	3	4	0	4	7	3	5	6	0	4	7	3	4	5
Physical frame 0	1	1	1	1	4	4	4	7	7	7	7	7	7	7	7	7	5
Physical frame 1		0	0	0	0	0	0	0	0	0	0	0	4	4	4	4	4
Physical frame 2			5	5	5	5	5	5	5	5	6	6	6	6	6	6	6
Physical frame 3				3	3	3	3	3	3	3	3	3	3	3	3	3	3
Page faults	F	F	F	F	F			F			F		F				F
Victim frames					1			4			5		0				7

ALGORITMUL OPTIMAL

- Nu prezintă anomalia lui Bélády

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	1
	2	2	2	2	2	2	2	2	3	4	4
		3	4	4	4	5	5	5	5	5	5

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	1	1	1	1	1	1
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	3	3	3	3	4	4
			4	4	4	5	5	5	5	5	5

ALGORITMUL LRU

○ Least Recently Used

- Folosim cunoștințele precedente – comportamentul de până acum
- Înlocuim pagina referită cel mai demult
- Pe baza principiului localității aceasta are probabilitatea cea mai mică să fie referită în viitorul apropiat
- Implementare dificilă
 - Etichetarea paginilor cu timpul ultimului acces
 - Încărcare semnificativă
 - Stivă de pagini – listă dublu înlănțuită
 - Pagina referită
 - Se mută în vârful stivei
 - Nu necesită căutare
 - Actualizare complicată
- LRU și Optimal
 - algoritmi bazați pe folosirea unei stive
 - nu prezintă anomalia lui Bélády

ALGORITMUL LRU

○ Exemplu 2

Reference string	1	0	5	3	4	0	4	7	3	5	6	0	4	7	3	4	5
Physical frame 0	1	1	1	1	4	4	4	4	4	4	6	6	6	6	3	3	3
Physical frame 1		0	0	0	0	0	0	0	0	5	5	5	5	7	7	7	7
Physical frame 2			5	5	5	5	5	7	7	7	7	0	0	0	0	0	5
Physical frame 3				3	3	3	3	3	3	3	3	3	4	4	4	4	4
Page faults	F	F	F	F	F			F		F	F	F	F	F	F		F
Victim frames					1			5		0	4	7	3	5	6		0

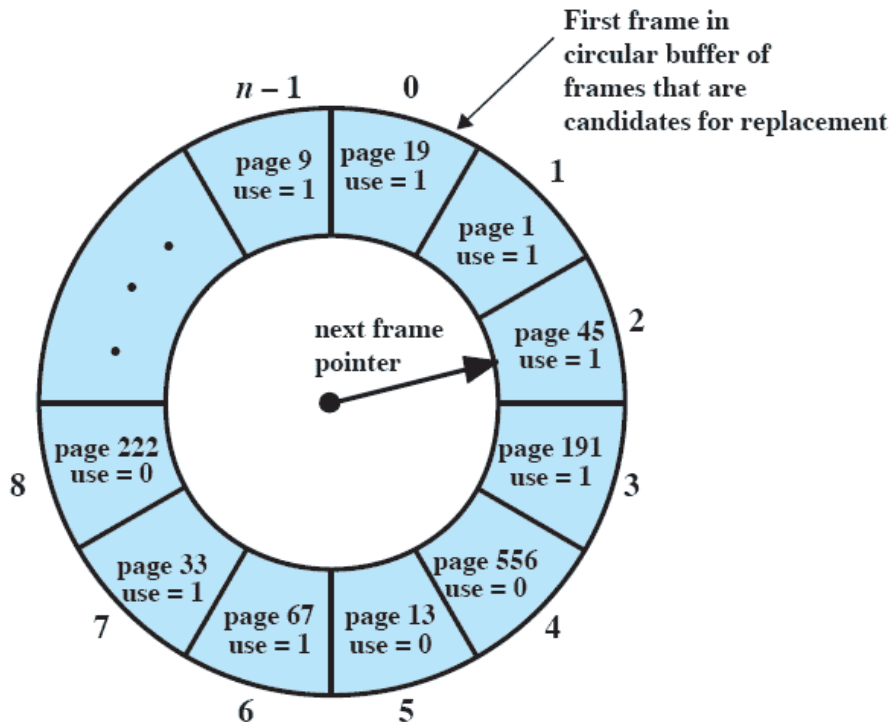
7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7 ₀	7 ₁	7 ₂	2 ₀	2 ₁	2 ₂	2 ₃	4 ₀	4 ₁	4 ₂	0 ₀	0 ₁	0 ₂	1 ₀	1 ₁	1 ₂	1 ₀	1 ₁	1 ₂	1 ₀
	0 ₀	0 ₁	0 ₂	0 ₀	0 ₁	0 ₀	0 ₁	0 ₂	3 ₀	3 ₁	3 ₀	3 ₁	3 ₂	3 ₃	0 ₀	0 ₁	0 ₂	0 ₀	0 ₁
		1 ₀	1 ₁	1 ₂	3 ₀	3 ₁	3 ₂	2 ₀	2 ₁	2 ₂	2 ₃	2 ₀	2 ₁	2 ₀	2 ₁	2 ₂	7 ₀	7 ₁	7 ₂

ALGORITMUL CLOCK

- Necesită asocierea unui bit adițional la fiecare cadru de pagină
 - Bit de utilizare
 - Evită eliminarea unei pagini intens utilizate
- La prima încărcare în memorie bitul de utilizare este setat la 1
 - Fiecare accesare care regăsește pagina în memorie setează bitul la 1
- Setul de cadre de pagină alocate sunt considerate ca și un buffer circular
 - Dispuse în cerc
- Selectarea unei pagini victimă
 - Dacă bitul de utilizare este 0
 - Pagina poate fi înlocuită
 - Dacă bitul de utilizare este 1
 - Schimbă valoarea la 0, dar lasă pagina în memorie
 - Caută altă pagină victimă

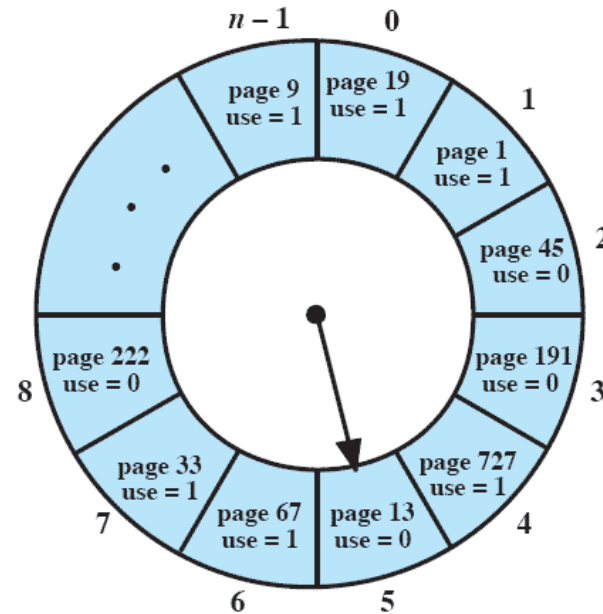
ALGORITMUL BAZAT PE CEAS

Stallings, pg. 366



(a) State of buffer just prior to a page replacement

Noua pagină: 727



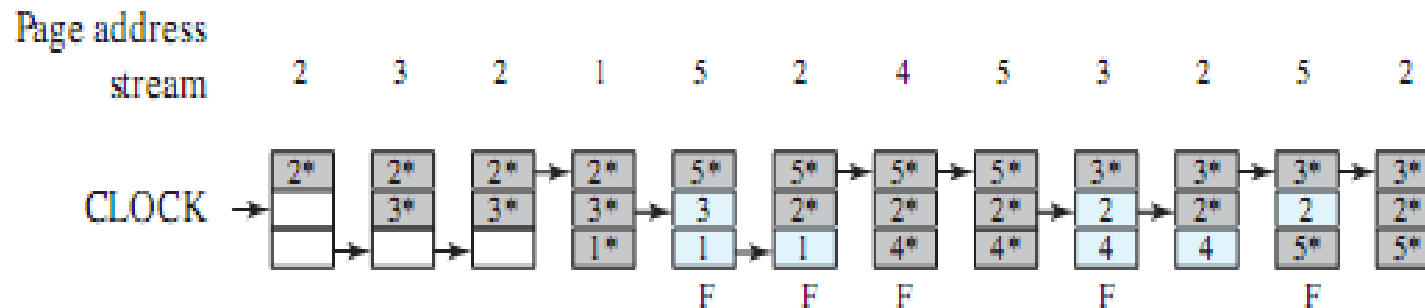
(b) State of buffer just after the next page replacement

Figure 8.16 Example of Clock Policy Operation

ALGORITMUL BAZAT PE CEAS

○ Exemplu

Stallings, pg. 364



F = page fault occurring after the frame allocation is initially filled

Figure 8.15 Behavior of Four Page Replacement Algorithms

REZULTATE COMPARATIVE

Stallings, pg. 367

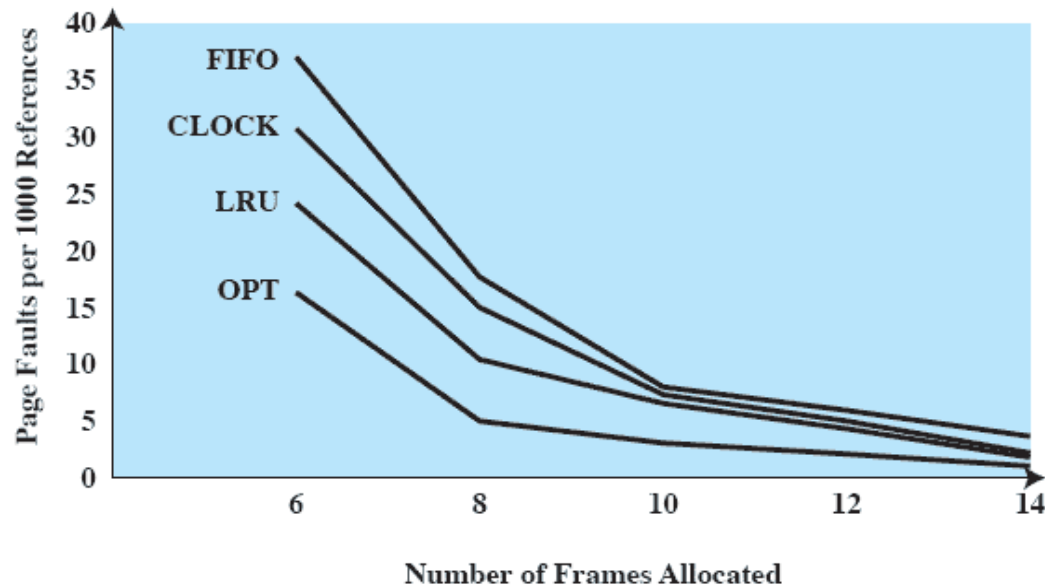


Figure 8.17 Comparison of Fixed-Allocation, Local Page Replacement Algorithms

ALGORITMI CU CONTORIZARE

- Mențin un contor care arată numărul de referiri la fiecare dintre pagini
 - Rar folosiți
- LFU – Least Frequently Used
 - Înlocuiește pagina referită cel mai puțin frecvent (cel mai rar)
 - cu cea mai mică valoare a contorului
- MFU – Most Frequently Used
 - Înlocuiește pagina cea mai des referită
 - Se bazează pe faptul că pagina cu foarte puține referiri este nou încărcată și urmează să fie folosită

GESTIUNEA SETULUI REZIDENT

- SO trebuie să decidă **câte cadre de pagină** alocă unui proces
 - Număr mic
 - Crește gradul de multiprogramare
 - pot coexista mai multe procese
 - Crește rata de page fault-uri
 - Peste o anumită limită
 - Creșterea numărului nu afectează rata de page fault-uri
- Alocare **fixă**
 - SO alocă un număr fix de cadre de pagină fiecărui proces
 - La page fault una din pagini va fi înlocuită
- Alocare **variabilă**
 - Permite ca numărul de cadre alocate unui proces să varieze în timp
 - Dacă apar multe page fault-uri SO poate decide să extindă numărul de cadre alocate
 - Soluție complicată
 - SO trebuie să evalueze comportamentul proceselor active

DOMENIUL DE ACȚIUNE AL POLITICII DE ÎNLOCUIRE

- Local

- Selectarea unei pagini victimă ia în considerare doar cadrele de pagină alocate procesului – din setul rezident

- Global

- Selectarea unei pagini victimă ia în considerare toate cadrele de pagină neblocate

- Ambele domenii sunt active dacă nu mai există cadre libere

GESTIUNEA SETULUI REZIDENT - REZUMAT

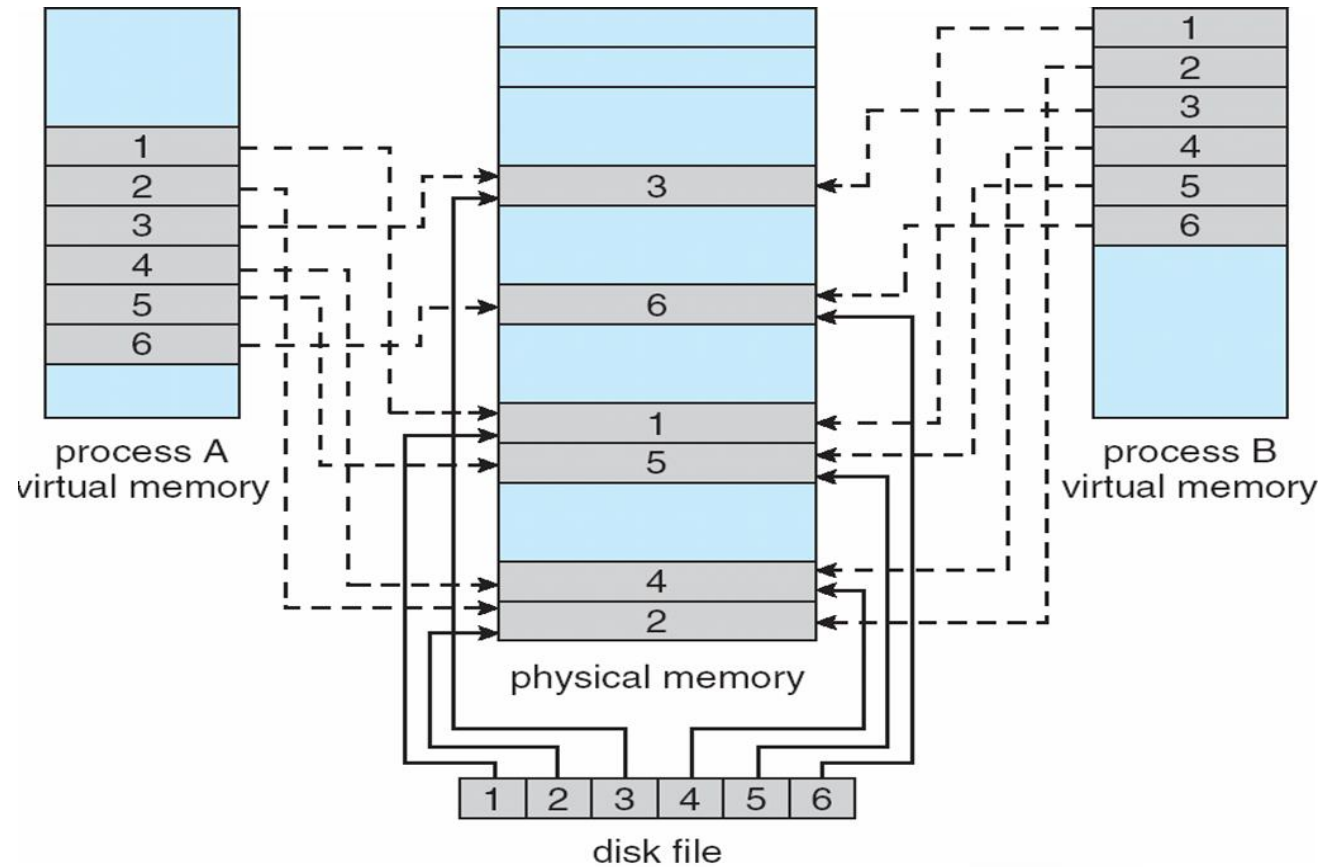
	Local Replacement	Global Replacement
Fixed Allocation	•Number of frames allocated to a process is fixed. •Page to be replaced is chosen from among the frames allocated to that process.	•Not possible.
Variable Allocation	•The number of frames allocated to a process may be changed from time to time to maintain the working set of the process. •Page to be replaced is chosen from among the frames allocated to that process.	•Page to be replaced is chosen from all available frames in main memory; this causes the size of the resident set of processes to vary.

FIȘIERE MAPATE ÎN MEMORIE

FIȘIERE MAPATE ÎN MEMORIE

- Fișierul sau parte din fișier mapat într-un set de pagini
- Operațiile de I/E asupra fișierelor mapate în memorie sunt tratate ca operații de acces la memorie
 - Scrierea la o adresă din memorie - scrierea în fișier
 - Scrierile înapoi pe disc nu sunt imediate
 - Se efectuează periodic sau
 - Testarea paginilor modificate – pagina victimă
 - Memoria joacă rolul unui cache
- Eficiență mult mai mare a operațiilor de I/E
- Inițial fișierul este citit folosind paginare la cerere
- Permite mai multor procese să mapeze același fișier – pagina / paginile din memorie sunt partajate

FIȘIERE MAPATE ÎN MEMORIE



POLITICA DE CURĂȚARE

POLITICA DE CURĂȚARE

- Determinarea momentului în care paginile modificate sunt scrise înapoi pe suportul din memoria secundară (discul)
- Două tipuri
 - Curățare la cerere
 - Pagina este scrisă înapoi doar când este selectată pentru înlocuire
 - Precurățare
 - Pagina este scrisă pe disc înainte să fie selectată ca pagină victimă
 - Permite scrierea în loturi (batch)

CONTROLUL ÎNCĂRCĂRII

CONTROLUL ÎNCĂRCĂRII

- Determină numărul de procese care vor fi rezidente în memoria internă
 - Gradul de multiprogramare
- Foarte important în gestiunea eficientă a memoriei
 - Număr mic de procese
 - Grad mic de utilizare a procesorului
 - Posibilitate mare ca toate procesele să fie blocate în operații de I/E
 - Swapping frecvent pentru a rula alte procese
 - Prea multe procese
 - Duce la thrashing

Stallings, pg. 377

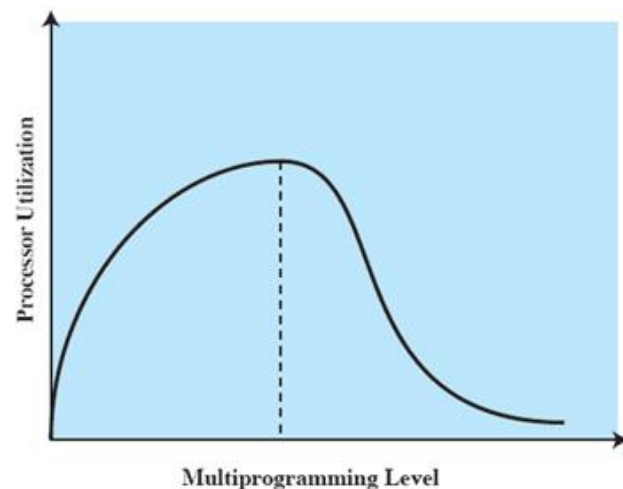


Figure 8.21 Multiprogramming Effects

CONCLUZII

○ Memoria virtuală

- Permite programelor să se execute chiar dacă spațiul de adrese nu este complet rezident în memoria fizică
- Necesită suport hardware și software
 - Suport hardware: MMU și TLB
 - Suport SO: sistem de memorie virtuală care controlează MMU și TLB

○ Cerințe pentru gestiunea memoriei

- Protecție
- Maparea eficientă a adreselor
 - Acces rapid la memorie, indiferent de schema de protecție folosită
 - Fără a apela la SO la fiecare acces!
- Schimbare de context rapidă